Compilerbook

Rui Ueyama 氏による公開記事 `compilerbook` をもとに、内容を Typst 向けに再構成した学習用ノートです。既存公開部分は論旨と章立てを保ちながら書き直し、未完だった ステップ 29 以降 は新規に補完しています。

> **編集注記**
> この文書は元サイトの公開本文をそのまま写したものではありません。公開されている章立てと説明の流れを保ちつつ、Typst で読みやすいように再構成した版です。ステップ 29 以降 については、公開ページ上の未完部分を埋めるための新規続編を含みます。

目次

Contents

はじめに 8
- なぜ「作りながら学ぶ」のか 8
- 本書のねらい 8
- 本書の読み方 9
- 途中で手を止めないためのコツ 9
- 想定する読者と開発環境 10
- 原著とこの再構成版 10
- 本書全体のロードマップ 10

言語処理系 10
- ソースコードから実行まで 11
- コンパイラ、アセンブラ、リンカ 11
- ローダと実行時環境 11
- インタプリタとの違い 12
- 本書がコンパイラへ集中する理由 12

文法 12
- 文法が必要になる瞬間 12
- BNF と再帰的な定義 12
- 優先順位と結合性 13
- 文法があるとテストしやすい 13
- 構文木との関係 13

プッシュダウンオートマトン 13
- 有限オートマトンでは足りない理由 13
- スタックが必要になる場面 14
- 理論と実装の距離感 14

再帰下降構文解析 14
- 再帰下降とは何をする方法か 14
- 左再帰を避ける 14
- 先読みと `consume` 14
- 再帰下降を採用する理由 15
- 最初の目標 15

機械語とアセンブラ 15
CPU とメモリ 15
よく出てくるレジスタ 15
メモリとアドレス 16
フラグと条件分岐 16
スタックとは何か 16
プロローグとエピローグ 16
アセンブラとは 16
なぜ最初からオブジェクトファイルを出さないのか 17
C とそれに対応するアセンブラ 17
簡単な式の例 17
関数呼び出しを含む例 17
アセンブリを読む練習 18
デバッグで見るべき点 18
命令を読むときの観点 18
本章のまとめ 19
電卓レベルの言語の作成 19
ステップ 1: 整数 1 個をコンパイルする 19
実装のポイント 19
このステップで見える課題 19
ステップ 2: + と - を実装する 20
なぜこの段階を挟むのか 20
よくある雑な実装とその限界 20
ステップ 3: トークナイザを導入する 20
トークン列という考え方 20
トークナイザのデータ構造 21
パーサが楽になる 21
ステップ 4: エラー報告を改善する 21
最低限ほしい診断 21
エラー処理は後から足すと辛い 21
文法を導入する 21
文法を入れる意味 22
左再帰に注意する 22
構文木を作る 22
なぜ木が必要なのか 22
ノードの種類 22
スタックマシンとしてコード生成する 22
一時値をどこへ置くか 23
ステップ 5: * と / を実装する 23
優先順位が実装へ現れる瞬間 23
除算で見えてくること 23
ステップ 6: 単項 + と - 23
どこで曖昧さが生まれるか 23
ステップ 7: 比較演算を追加する 23
文法上の位置 24
コード生成の変化 24

ここまでのテストケース 24
前半の設計で残してよい粗さ 24
この段階で見えてくること 25
分割コンパイルとリンク 25
オブジェクトファイルとは何か 25
再配置情報の意味 25
ライブラリと再利用 25
静的ライブラリと共有ライブラリ 26
関数とローカル変数 26
ステップ 8: 文を読む 26
文と式の違い 26
ステップ 9: 1 文字のローカル変数 26
左辺値という概念 26
ステップ 10: `return` 27
なぜ共通エピローグにするのか 27
戻り値レジスタの存在 27
ステップ 11: 複数文字のローカル変数 27
予約語との区別 27
ステップ 12: 制御構文 27
制御構文は「文脈」を持つ 27
`for` 文が持つ複数の場所 27
ステップ 13: ブロック 28
ステップ 14: 関数呼び出し 28
スタックアラインメントの注意 28
評価順序の感覚 28
ステップ 15: 関数定義 28
関数ごとの独立性 28
x86-64 の呼び出し規約 28
コンピュータにおける整数の表現 29
符号なし整数 29
符号付き整数と 2 の補数 29
符号拡張 29
否定とオーバーフロー 29
制御構文のコード生成を読む例 29
8 ビット整数で考える練習 30
この章の位置づけ 30
ポインタと文字列リテラル 30
ステップ 16: 単項 `&` と `*` 30
`&` の意味 31
`*` の意味 31
左辺値を扱う内部関数 31
ステップ 17: `int` キーワード 31
型名が文法へ入る 31
ステップ 18: ポインタ型 31
型データ構造が必要になる理由 31
型の大きさ 32

左辺値と右辺値の整理 32
ステップ 19: ポインタ加算 32
なぜスケーリングが必要なのか 32
引き算や比較への布石 32
ステップ 20: sizeof 32
sizeof が教えてくれること 32
式と型の両方を取れる 32
ステップ 21: 配列 33
型の複雑化 33
配列の配置 33
配列とポインタ 33
ステップ 22: 添字演算子 [] 33
表面構文を既存要素へ還元する 33
i[a] が合法な理由 33
ステップ 23: グローバル変数 33
トップレベルの意味が増える 33
スコープ感覚の変化 34
ステップ 24: char 34
小さい整数型が持ち込む問題 34
なぜ char が早めに必要なのか 34
ステップ 25: 文字列リテラル 34
コードではなくデータを生成する 34
文字列リテラルの一意なラベル 34
ステップ 26: ファイル入力 34
行番号とファイル名 34
入力インターフェースの変化 34
ステップ 27: コメント 35
地味だが効く改善 35
トークナイザ責務の確認 35
ステップ 28: テストを C で書き直す 35
なぜ大きな節目なのか 35
テストコードが仕様書になる 35
プログラムの実行イメージと初期化式 35
実行ファイルの構造 35
テキスト、データ、BSS 35
グローバル変数の初期化式 36
なぜ定数式に限られるのか 36
実装方針 36
ローカル変数の初期化 36
見た目は似ていても意味は違う 36
ポインタと配列の確認例 36
初期化式でよくあるつまずき 37
この章の位置づけ 37
ステップ 29 以降 37
ここから先の難しさ 37
ステップ 29: 構文木に型を載せる 37

なぜ最初に型なのか 38
どこで型を付けるか 38
必要になる型データ 38
型エラー検出は後でもよい 38
ステップ 30: typedef と宣言子 38
何が難しいのか 38
宣言子が複雑になる理由 38
typedef 名のスコープ 39
ステップ 31: struct とメンバアクセス 39
構造体型が必要な理由 39
実装上の中心はメンバ表 39
パディングとアラインメント 39
. と -> 39
ステップ 32: enum、switch、case 40
enum の意義 40
switch が欲しくなる理由 40
実装の骨格 40
ステップ 33: break、continue、goto 40
文脈付きジャンプ 40
goto の前方参照 40
なぜ goto を入れるのか 41
ステップ 34: キャストと void 41
パース上の難しさ 41
void の最小実装 41
変換命令の実際 41
ステップ 35: 複合代入と前置・後置インクリメント 41
何が危険なのか 41
実装の中心 41
前置と後置の違い 41
ステップ 36: 関数宣言と型チェックの強化 42
ここからの主題 42
関数プロトタイプ 42
暗黙変換と整合性 42
ここまで実装できると何が嬉しいのか 42
その先に残るもの 42
後半機能のテスト戦略 42
スタティックリンクとダイナミックリンク 43
静的リンク 43
動的リンク 43
コンパイラとの関係 43
C の型の構文 43
なぜ読みづらいのか 43
宣言子を再帰的に読む 43
実装上の利点 44
読み方の練習 44
おわりに 44

付録 1: x86-64 命令セット チートシート 44
整数演算 44
メモリアクセス 44
分岐と条件コード 45
関数呼び出し 45
覚え方のコツ 45
付録 2: Git によるバージョン管理 45
おすすめのコミット単位 45
履歴が仕様書になる 45
付録 3: Docker を使った開発環境の作成 46
なぜ環境固定が効くのか 46
最小構成の考え方 46
付録 4: テストケースの育て方 46
付録 5: ステップ別サンプルプログラム 46
式だけの段階 46
文、変数、関数の段階 47
ポインタ、配列、グローバル変数の段階 48
型と後半機能の段階 48
付録 6: 実装チェックリスト 49
字句解析 49
構文解析 49
構文木と型 50
コード生成 50
ビルドと検証 50
付録 7: デバッグの進め方 50
字句解析で疑うべきこと 50
構文解析で疑うべきこと 50
型と意味解析で疑うべきこと 51
コード生成で疑うべきこと 51
失敗する入力を小さくする 51
付録 8: セルフホストへ向かう順序 51
まず必要なもの 51
ブートストラップの考え方 51
セルフホストで露呈しやすい弱点 52
無理なく近づくために 52
付録 9: よくあるつまずきと考え方 52
パーサが壊れて見えるが、実はトークナイザの問題 52
生成コードが壊れて見えるが、実は左辺値処理の問題 52
構文は通るが意味が変になる 52
ある入力では動くが、少し長い入力で急に壊れる 53
エラー報告が弱いと進まなくなる 53
設計を途中で作り直すのは普通 53
付録 10: 章ごとの復習ポイント 53
導入と理論 53
式だけのコンパイラ 53
文、変数、関数 53

ポインタと配列 ... 54
後半の型と内部表現 ... 54
その先へ進むために ... 54
付録 11: 用語小事典 ... 54
AST ... 54
字句解析 ... 54
構文解析 ... 54
意味解析 ... 54
左辺値 ... 54
右辺値 ... 55
スコープ ... 55
ABI ... 55
再配置 ... 55
セルフホスト ... 55
宣言子 ... 55
暗黙変換 ... 55
パディング ... 55
スタックフレーム ... 55
トークン ... 55
予約語 ... 56
オフセット ... 56
テキストセクション ... 56
データセクション ... 56
BSS ... 56
プロローグ ... 56
エピローグ ... 56
フォールスルー ... 56
変換単位 ... 56
デケイ ... 56
シンボル ... 57
付録 12: 追加練習課題 ... 57
課題 1: AST ダンプを整える ... 57
課題 2: エラーメッセージへ行番号を入れる ... 57
課題 3: `for` を `while` へ展開して考える ... 57
課題 4: 構造体のアラインメントを可視化する ... 57
課題 5: `switch` のテストを増やす ... 57
課題 6: 複合代入の副作用を確認する ... 57
課題 7: 既存 C コードの小片を通す ... 57
課題 8: 型エラーを少し厳密にする ... 57
課題 9: セルフホスト準備として構造体を増やす ... 57
課題 10: 実装履歴を説明資料にする ... 58
付録 13: よくある質問 ... 58
なぜ最初から最適化を考えないのですか ... 58
なぜアセンブリ出力なのですか ... 58
なぜ前半では型チェックを厳密にしないのですか ... 58
ポインタはなぜ難しく感じるのですか ... 58

構造体を入れると何が嬉しいのですか 58
セルフホストは必須ですか 58
実装が汚くなってきたらどうすればよいですか 58
何を読めるようになれば次へ進んでよいですか 58
付録 14: 次に読むなら 59
参考資料 59
索引 59

はじめに

この文書は、小さな C コンパイラを少しずつ育てながら、言語処理系の全体像を具体的に理解することを目的にしたノートです。コンパイラという言葉にはどこか近寄りがたい響きがありますが、実際にやっていることは「入力を読み、構造を見抜き、別の表現へ変換する」ことに尽きます。巨大な仕様書を読む前に、動くものを作って眺めるほうが本質に近づきやすい分野です。

本書では、常に動くコンパイラを保ちながら、機能を 1 歩ずつ追加していきます。最初は整数 1 個しか読めないような極小の処理系から始め、四則演算、変数、制御構文、関数、ポインタ、配列、グローバル変数、型と段階的に伸ばしていきます。重要なのは、毎回の変更が小さいことです。変更が小さければ、なぜその設計が必要になったのかを自分の目で追えますし、壊れたときにも原因が見つけやすくなります。

なぜ「作りながら学ぶ」のか

コンパイラの説明で難しく感じやすいのは、字句解析、構文解析、意味解析、コード生成、リンカ、実行形式といった要素が最初から全部並んで見えてしまうからです。個々はそれほど複雑ではなくても、最初に同時に見ると「結局どこから手を付ければよいのか」が見えなくなります。

作りながら学ぶ方法には、この問題を強引に解決する力があります。たとえば「整数 1 個を返すだけの関数」をコンパイルする段階では、考えるべきことは入力を整数として解釈し、それを `mov` と `ret` に落とすことだけです。この小さな成功体験の上に、次は `+`、その次は `*`、という形で理解を積み上げられます。

さらに、実装を伴う学習には、抽象概念に身体感覚が生まれるという利点があります。字句解析器の責務は何か、構文木を作る利点は何か、左辺値を分けて扱う必要はどこから出てくるのか、といった話は、実際に 1 度コードにすると急に具体的になります。本書はまさにその瞬間を積み重ねる構成です。

本書のねらい

本書が最終的に目指すのは、完全な C 規格処理系ではありません。C の全仕様を厳密に実装しようとすると、プリプロセッサ、浮動小数点数、複雑な ABI、さまざまな最適化など、一気に話が大きくなります。本書の中心にあるのは、そうした巨大な全体像の前に、コンパイラの骨格を自力で作れるようになることです。

その骨格とは、もう少し具体的に言えば次のようなものです。

- 入力をトークン列へ分解する字句解析器
- 文法に従ってトークン列を読む構文解析器
- 構造を保持する抽象構文木
- 必要な型情報やスコープ情報

- x86-64 向けに命令列を出力するコード生成器

ここまで自分で作れるようになると、コンパイラ本や実装記事を読むときの解像度が一気に上がります。たとえば「左再帰を除去する」「ポインタ加算では要素サイズを掛ける」「構造体メンバにオフセットを付ける」といった説明が、単なる知識ではなく具体的な実装手順として理解できるようになります。

もうひとつの狙いは、コンパイラを通じて周辺技術も学べることです。アセンブラは何をしているのか、リンカはどこで必要になるのか、オブジェクトファイルには何が入っているのか、実行ファイルはどうやってメモリへ置かれるのか。これらは別々の話に見えて、実際には 1 本のパイプラインの中にあります。本書では、コンパイラだけを孤立した箱として扱わず、その前後にある世界も見えるように進めます。

本書の読み方

最も効果的な読み方は、章を読むたびに自分の実装へ同じ変更を入れることです。コンパイラ本は文章だけ読んでも面白いのですが、理解が深くなるのは「これを自分ならどう書くか」と考え始めた瞬間です。字句解析器でもコード生成器でも、実際に 1 度書くと設計の理由が急に見えます。

読んでいて途中で詰まりやすいのは、細部を完全に理解してから先へ進もうとする場合です。実際には、各ステップで必要なことだけを押さえて進んだほうが全体像は掴みやすくなります。たとえば最初の段階では、レジスタ割り当ての最適化や厳密な型エラー検出まで気にする必要はありません。今の段階で必要な最小限だけを作り、後で差し替えることが前提です。

章の途中に出てくるコード例は、必ずしもそのまま実装へ貼り付けることを意図していません。説明の中心にある考え方を示すためのものです。実際にはエラー処理、ヘルパ関数、テストコード、デバッグ用出力などがもう少し必要になるでしょう。それでも、機能追加の順番だけはあまり崩さないほうが理解しやすくなります。前のステップで導入した内部表現が、次のステップの土台になるように設計しているからです。

途中で手を止めないためのコツ

コンパイラ作成は、勢いよく数日で完成する種類の作業ではありません。少しずつ機能を増やし、少しずつテストを増やし、時々設計を整理し直しながら進みます。途中で止まりにくくするためには、毎回の作業単位を小さく保つのが重要です。

おすすめは、1 つのステップをさらに細かく分けることです。たとえば `if` を実装するなら、まず予約語認識、次に構文木ノード追加、次にコード生成、最後にテスト、という具合に分けます。`struct` を実装するなら、型定義、メンバ表、オフセット計算、`.` 演算子、`->` 演算子、`sizeof` の修正、という形に分けられます。小分けにしておけば、毎回の差分が読みやすくなり、壊れたときも戻りやすくなります。

もうひとつ大切なのは、途中結果を可視化することです。トークナイザを書いたらトークン列をダンプする、パーサを書いたら構文木を簡単に表示する、コード生成を書いたら生成されたアセンブリを読む。コンパイラは内部表現が多いので、目に見える形で中間状態を確認できるだけでデバッグが格段に楽になります。

想定する読者と開発環境

想定している読者は、C か C++である程度プログラムを書いたことがあり、変数、関数、ポインタの基本は理解している人です。アセンブリや ELF 形式に詳しい必要はありません。本書の前半で必要になる低レイヤの知識は、その都度導入します。

環境としては、C コンパイラ、`make`、標準的な Unix 系ツールが使える Linux または macOS を想定します。Windows でも WSL やコンテナを使えば問題ありません。最初は `gcc` か `clang` が入っていれば進められますし、確認用に `objdump`、`readelf`、`nm` のようなツールがあると理解が深まります。

本書の実装は説明しやすさを優先しているため、最初のうちは性能や完全性を追いません。生成するコードが多少冗長でも、内部構造が明快であればそれで十分です。あとから改善できるところを最初から全部良くしようとすると、かえって何も進まなくなります。

原著とこの再構成版

原著の `compilerbook` は Rui Ueyama 氏が公開している教材です。同氏は 9cc や chibicc の開発でも知られており、小さな実装を段階的に育てる説明が非常にわかりやすいことで有名です。この文書は、その公開構成と論旨を踏まえつつ、Typst で読み通しやすいように再構成した版です。

したがって、ここに書かれている文章は公開サイト本文の逐語転記ではありません。見出しや流れを保ちながら、自分の言葉で説明し直し、未完だった後半については実装方針が追える形で補っています。その意味では、原著の導線を尊重しつつ、少し長めの学習ノートとして読み進めるのが自然です。

本書全体のロードマップ

本書を読み進めるときは、全体を次の 4 段階だと思うと整理しやすくなります。

- 第 1 段階: 整数式だけを扱う最小コンパイラを作る
- 第 2 段階: 文、変数、関数を入れてプログラムらしくする
- 第 3 段階: ポインタ、配列、グローバル変数で C らしさを強める
- 第 4 段階: 型、宣言子、構造体、制御構文を強化して後半へ進む

この見方をしておくと、各ステップが孤立した小技ではなく、「どの段階のための変更なのか」が見えやすくなります。前半で作る字句解析器や構文木は、後半の難しい機能に備える土台です。後半で苦労して導入する型やスコープは、前半で動いたものを壊すためではなく、より大きなプログラムを受け止めるために必要になります。

また、各段階の終わりで小さな節目を意識すると、学習が続きやすくなります。第 1 段階の終わりでは四則演算の式が通ること、第 2 段階では関数と制御構文が通ること、第 3 段階ではポインタと配列が使えること、第 4 段階では構造体や `typedef` が動くこと、という具合に、自分の実装の到達点をはっきりさせておくと前進を実感しやすくなります。

言語処理系

コンパイラは、言語処理系の一部です。人間が読み書きする高級言語と、CPU が直接理解する命令列のあいだには、いくつもの層があります。ソースコードをそのまま機械語へ落とすだけではなく、アセンブラ、リンカ、ローダ、ライブラリが協調して初めて 1 つのプログラムが動きます。

ソースコードから実行まで

まずは全体の流れを非常に単純化して眺めてみましょう。

```
ソースコード
  ↓
コンパイラ
  ↓
アセンブリ or オブジェクトファイル
  ↓
アセンブラ
  ↓
オブジェクトファイル
  ↓
リンカ
  ↓
実行ファイル
  ↓
ローダ
  ↓
プロセスとして実行
```

この流れを最初に頭へ入れておくと、自分が今どこを作っているのかが見失いにくくなります。たとえば本書の前半では主に「ソースコードからアセンブリを出すまで」を扱います。アセンブラやリンカは既存の道具に任せ、コンパイラ自身がどこまで責任を持つべきかを整理しながら進みます。

コンパイラ、アセンブラ、リンカ

典型的な C プログラムは、まずコンパイラによってアセンブリやオブジェクトファイルへ変換されます。そのあとアセンブラがテキスト形式の命令列を機械語へ変換し、リンカが複数のオブジェクトファイルとライブラリを 1 個の実行形式へまとめます。

この分業には実際的な理由があります。コンパイラは言語の意味を解釈するのが主な仕事で、シンボル解決や最終的な配置はリンカが担当したほうが都合がよいのです。逆に言えば、コンパイラ学習の最初から実行形式全体を自力で組み立てようとすると、言語処理の本質とは別の問題に大量の注意を奪われます。

たとえば `printf` を呼び出すプログラムを考えると、コンパイラが知っていればよいのは「この名前の関数を呼ぶ」という事実までです。`printf` の実アドレスがどこにあるかは、最終的にリンカやローダが決めます。この役割分担を知っているだけで、コンパイラ実装の視界はかなりすっきりします。

ローダと実行時環境

実行ファイルができても、それだけでは CPU はまだ動きません。OS のローダが実行形式を読み込み、コードやデータを適切なメモリ位置へ配置し、スタックを用意し、プログラムの入口へ制御を渡して初めて実行が始まります。

本書ではローダ自体を実装するわけではありませんが、こうした前後の仕組みを薄く知っておくと、たとえばグローバル変数や文字列リテラルがどこに置かれるのか、なぜ関数呼び出し規約が必要なのか、といった話が自然に理解できます。コンパイラは決して孤立した存在ではありません。

インタプリタとの違い

同じ「プログラムを動かす」仕組みでも、インタプリタとコンパイラでは役割が異なります。インタプリタは入力を読みながら逐次評価するのに対し、コンパイラは一度別の形式へ変換し、その生成物を後で実行します。

実装の見通しも変わります。インタプリタでは式木をその場で評価すれば形になりますが、コンパイラでは「CPU が実行可能な命令列」を出力しなければなりません。したがって、メモリ配置やレジスタ、呼び出し規約まで視野に入ってきます。

もっとも、両者に共通するところもあります。どちらも字句解析や構文解析が必要ですし、構文木を作れば実装しやすくなる点も似ています。その意味で、コンパイラ作成は言語実装全般の良い入口でもあります。

本書がコンパイラへ集中する理由

教材として何を中心に据えるかを考えたとき、コンパイラはちょうどよい題材です。入力言語も出力言語も明確で、途中の中間表現が多く、テストもしやすいからです。しかも、作りながらアセンブリ、オブジェクトファイル、ABI、実行形式といった周辺知識まで自然に学べます。

もちろん、現実の処理系はインタプリタや JIT、バイトコード仮想機械など多様です。それでも、構文を理解し、内部表現へ落とし込み、対象実行系に合わせて変換するという流れは共通しています。本書で得る感覚は、他の言語実装へ移っても十分に役立ちます。

文法

コンパイラ作成を始めると、すぐに「この文字列はどのように構造化されているのか」を曖昧なく表したくなります。そのための道具が文法です。文法は、ある言語で許される文字列の集合を再帰的に定義する仕組みだと考えるとわかりやすいでしょう。

文法が必要になる瞬間

たとえば四則演算を含む式を処理したいとして、単に `+` や `*` を見つければよいわけではありません。`1+2*3` を正しく読むには、掛け算が足し算より先に結びつくことを表現しなければなりません。`(1+2)*3` のように括弧が入ると、さらに「どこまでが 1 つの部分式か」を区別する必要があります。

このようなとき、文字列をその場で場当たり的に読んでいると、少し複雑な入力でもすぐに破綻します。文法を先に置いておけば、「どの規則で何を読んでいるのか」がはっきりし、パーサの関数分割も自然に決まります。

BNF と再帰的な定義

文法の記法としてよく使われるのが BNF やその拡張記法です。たとえば次のように書けます。

```
expr    = mul (("+" | "-") mul)*
mul     = unary (("*" | "/") unary)*
unary   = ("+" | "-")? primary
primary = num | "(" expr ")"
```

ここで重要なのは、文法が再帰的であることです。`primary` の中に `expr` が入っているので、括弧式の中にさらに式を入れられます。再帰の力があるからこそ、言語は有限個の規則で非常に多くの入力を表現できます。

優先順位と結合性

文法は、単に「どんな文字列を受理するか」だけではなく、「どのように結びつけて読むか」も表現します。足し算と掛け算の優先順位が典型例です。`expr` の中から `mul` を呼ぶ形にしておけば、掛け算が足し算より 1 段強く結びつくことが自然に実装できます。

結合性も同じです。`1-2-3` を `(1-2)-3` と読むのか、`1-(2-3)` と読むのかは文法で決まります。本書のような通常の算術式では左結合にするのが自然です。こうした規則が曖昧なままだと、構文木を 1 つに定められず、その後のコード生成も定まりません。

文法があるとテストしやすい

文法を明示すると、受理すべき入力と拒否すべき入力を整理しやすくなります。たとえば `1+2*3` は受理、`1+*3` は拒否、`((1))` は受理、`(1+2` は拒否、といった具合です。これにより、パーサのテストケースが非常に作りやすくなります。

また、エラーが出たときに「どの非終端記号を読んでいる途中だったか」がわかるだけで、診断文も改善しやすくなります。単に「構文エラー」と言うのではなく、「ここでは式の終端が必要だが `*` が来た」のように、より具体的な情報を返せるようになります。

構文木との関係

文法に従って入力を読むと、その結果を木構造として表したくなります。これが構文木です。たとえば `1+2*3` は、根が `+` で、右の子が `*` になっている木として表現できます。

```
    (+)
   /   \
 (1)    (*)
       /   \
    (2)    (3)
```

構文木の利点は、入力文字列を読み終えたあとに、構造を見失わずに済むことです。コード生成、型付け、定数式評価、最適化など、後続の処理はほとんどすべて木構造の上で考えたほうが整理しやすくなります。

プッシュダウンオートマトン

有限オートマトンだけでは、括弧の対応のような入れ子構造をうまく扱えません。そこで必要になるのが、スタックを持つオートマトンです。プッシュダウンオートマトンという名前は少し大げさに見えますが、本質は「必要なところで情報を積み、あとで取り出せる」ことにあります。

有限オートマトンでは足りない理由

有限オートマトンは、現在状態が有限個しかない計算モデルです。これは字句解析のような比較的単純な問題にはよく向いていますが、括弧の深さを無制限に数える必要がある問題には向きません。`(` がいくつ開いたかを理論上どこまでも覚えるには、有限個ではない記憶が必要になるからです。

その典型例が、入れ子になった式や文です。`((1+(2*3)))` のような入力では、どの括弧がどの括弧に対応しているかを途中で覚えておく必要があります。これはまさにスタックの得意分野です。

スタックが必要になる場面

スタックは、最後に積んだものを最初に取り出す記憶構造です。関数呼び出しで戻り先を覚えておくとき、再帰的な構文解析で途中状態を保持するとき、式評価の一時値を退避するときなど、コンパイラ実装では何度も顔を出します。

たとえば再帰下降構文解析では、`primary()` が括弧式を読んだときに `expr()` を再び呼びます。これは理論的に見れば、「今どこまで読んでいたか」という途中状態を暗黙のスタックへ積んでいるのと同じです。プッシュダウンオートマトンの考え方は、そのような動作を抽象化したものだと言えます。

理論と実装の距離感

コンパイラ実装では、理論としてプッシュダウンオートマトンを厳密にコード化することはあまりありません。それでも、「入れ子を扱うには有限オートマトンだけでは足りず、スタックが必要になる」という感覚は非常に役に立ちます。字句解析と構文解析の境界を理解するときにも、この感覚が支えになります。

つまり、本書でこの話を出す理由は、理論用語を暗記してほしいからではありません。括弧、ブロック、関数呼び出し、再帰的型、ネストした制御構文を見たときに、「ここはスタックが効いてくる場面だ」と自然に思えるようになるためです。

再帰下降構文解析

本書で採用するパーサは再帰下降構文解析です。これは、文法の各非終端記号に対応する関数を 1 つずつ書き、関数呼び出しによって構文の再帰性を表現する方法です。

再帰下降とは何をする方法か

考え方は驚くほど素朴です。`expr` という規則があるなら `expr()` という関数を書く。`mul` という規則があるなら `mul()` を書く。関数の中で、対応するトークン列を読み、必要なら下位の関数を呼び、最後に構文木ノードを返します。

この素朴さが大きな利点です。文法とコードの対応がほぼ 1 対 1 になるため、説明しやすく、変更しやすく、壊れたときにも追いやすいのです。教材や小規模実装では、これ以上に見通しのよい方法はあまりありません。

左再帰を避ける

再帰下降で少しだけ注意が必要なのは、左再帰です。たとえば `expr = expr "+" num | num` のような文法をそのまま関数へ落とすと、`expr()` が最初に `expr()` を呼び、そこでまた `expr()` が呼ばれて無限再帰になります。

そこで、再帰下降に向いた形へ文法を書き換えます。本書で使っている `expr = mul (("+" | "-") mul)*` のような形は、その典型です。左再帰を排除しつつ、左結合の意味を保っています。理論の小さな工夫が実装の素朴さを支えている、よい例です。

先読みと `consume`

再帰下降パーサでは、多くの場合「今見ているトークンが何か」を確認しながら進みます。そのために、現在位置を指すポインタやインデックスと、`consume("+")` や `expect(TK_NUM)` のような補助関数を用意しておくと便利です。

こうした補助関数があると、パーサ本体は「どんな構文を読むか」に集中できます。失敗した場合のエラー報告も一箇所へ集めやすくなるので、実装の読みやすさがかなり改善します。

再帰下降を採用する理由

LALR や LR パーサ生成器を使う方法もありますが、最初の教材としては内部の見通しが悪くなりがちです。再帰下降なら、`expr()` が式を、`mul()` が乗除算を、`primary()` が整数や括弧式を読む、と関数の責務がそのまま文法の責務に対応します。

また、途中で構文木ノードをその場で作りやすいのも大きな利点です。本書のように機能を少しずつ足すスタイルでは、手書きの再帰下降のほうが圧倒的に改造しやすく、失敗したときのデバッグもしやすくなります。

最初の目標

ここまでの話を踏まえると、本書の最初の数ステップでやることははっきりしています。まずは簡単な字句解析器を作り、再帰下降パーサで式を読み、構文木を作り、スタックマシン風にコード生成する。それだけです。

最初は驚くほど小さなプログラムですが、この小ささこそが重要です。コンパイラの骨格は、巨大な仕様からではなく、小さな成功の連鎖から見えてきます。次章では、その最初の足場として、機械語とアセンブラの最低限を押さえます。

機械語とアセンブラ

高級言語から始めると忘れがちですが、最終的に CPU が理解するのは機械語だけです。アセンブリは、その機械語を人間が読み書きしやすい記号へ置き換えた表記にすぎません。本章では、後のコード生成で必要になる最低限の見取り図を作ります。

コンパイラを書き始めたばかりの時期にアセンブリを見るのは少し怖く感じるかもしれません。しかし、最初に必要な知識は驚くほど少ないです。どこに値が置かれるのか、四則演算はどう表されるのか、関数呼び出しはどう見えるのか。この 3 点がわかれば、最初のコード生成器は十分に書けます。

CPU とメモリ

CPU はレジスタと呼ばれる小さく高速な記憶領域を持ち、命令は主にそこを読み書きしながら進みます。一方、メインメモリは容量は大きいものの遅いため、必要な値をレジスタへ読み込み、演算し、必要なら書き戻すという流れを取ります。

x86-64 では `rax`、`rdi`、`rsp`、`rbp` などの汎用レジスタが使われます。コンパイラを書く立場から重要なのは、どのレジスタが戻り値に使われるか、引数に使われるか、スタックを指すのはどれか、という点です。命令セットを全部暗記する必要はありませんが、役割の代表例は早めに知っておいたほうがよいです。

よく出てくるレジスタ

教材実装の前半で特によく見るのは次のようなレジスタです。

- `rax`: 戻り値や一時値に使われやすい
- `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`: 整数引数の受け渡しに使われる
- `rsp`: 現在のスタック先頭を指す
- `rbp`: スタックフレームの基準位置として使われることが多い

もちろん実際のコンパイラではもっと多くのレジスタを使いますが、最初の段階では「今計算中の値」と「スタックの状態」が追えれば十分です。スタックマシン風のコード生成を採るなら、途中結果を一時的に `push` / `pop` しながら進めればよいので、レジスタ割り当てを早い段階で悩む必要はありません。

メモリとアドレス

メモリは大きなバイト列で、各位置にアドレスが付いています。変数はどこかのアドレスに格納され、ポインタはそのアドレスそのものを値として持ちます。コンパイラがやっていることのかなりの部分は、「この値は今どのアドレスにあるか」「どの命令でそこへアクセスするか」を管理することだと言ってもよいでしょう。

ローカル変数は通常スタック上に置かれます。したがって `x` という変数を読むとき、実際には「`rbp` から何バイト離れた場所を読むか」という問題に落ちます。これがスタックフレームやオフセット計算の意味です。

フラグと条件分岐

CPU には、演算結果に応じて更新されるフラグもあります。比較命令や算術命令のあと、ゼロだったか、負だったか、桁あふれしたか、といった状態が内部に記録されます。その後の条件ジャンプは、このフラグを見て飛ぶかどうかを決めます。

コンパイラを書くとき、比較演算子をどう実装するかでこの仕組みが重要になります。`a < b` を評価したいなら、まず `cmp` のような命令で比較し、その結果に応じて `setl` や `jl` を使います。高級言語では見えない「比較した結果がしばらく CPU 内部に残っている」という感覚を持てると、条件分岐のコード生成が理解しやすくなります。

スタックとは何か

スタックは、後入れ先出しのデータ構造です。関数呼び出しでは復帰アドレスや一時値、ローカル変数の領域確保などに使われます。x86-64 では通常、スタックは高位アドレスから低位アドレスへ向かって伸びます。つまり `push` すると `rsp` が小さくなり、`pop` すると大きくなります。

この性質は、再帰下降構文解析と似たところがあります。途中状態を積み、必要になったら戻るという流れが繰り返し現れるからです。コンパイラ実装では「構文解析のための暗黙のスタック」と「生成コードが使う明示的なスタック」の両方が登場します。

プロローグとエピローグ

関数の先頭では、古いベースポインタを保存し、新しいスタックフレームを作る処理がよく入ります。これがプロローグです。逆に、関数の末尾ではスタックを元へ戻し、保存したベースポインタを復元して `ret` します。これがエピローグです。

典型的には次のような形になります。

```
push rbp
mov rbp, rsp
sub rsp, 32
```

これは「呼び出し元の `rbp` を保存し、今の `rsp` を新しいフレーム基準にし、ローカル変数ぶんの領域を確保する」という意味です。関数が終わるときには `mov rsp, rbp` と `pop rbp` で元の状態へ戻します。

アセンブラとは

アセンブラは、`mov rax, 3` のようなテキスト形式の命令を機械語へ変換するプログラムです。人間が生のバイト列で命令を書くのは現実的ではないため、アセンブリという中間表現が用意されています。

コンパイラ学習では、最初から機械語を直接出すより、アセンブリを出すほうが圧倒的に楽です。人間が目で読めて、間違いがあれば `objdump` などで確認できるからです。生成物をそのまま眺められるというのは、デバッグ上きわめて大きな利点です。

なぜ最初からオブジェクトファイルを出さないのか

もちろん、コンパイラが直接オブジェクトファイルを吐くことも理屈の上では可能です。しかしその場合、命令エンコーディング、再配置情報、セクション構造などをかなり早い段階から意識しなければなりません。教材の最初の数ステップとしては重すぎます。

一方、アセンブリを出力する形にしておけば、コンパイラの責務は「この意味を持つ命令列を組み立てる」ことに集中できます。アセンブラが生の機械語化を引き受けてくれるため、学習の焦点がぶれません。

C とそれに対応するアセンブラ

単純な C コードからアセンブリへの落ち方を見ておくと、後のコード生成が理解しやすくなります。たとえば次のような関数を考えます。

```
int main() {
  return 3;
}
```

この関数の本質は、「戻り値レジスタへ 3 を入れて戻る」ことだけです。実際のアセンブリはプラットフォームや最適化設定で多少変わりますが、概ね次のような形になります。

```
main:
  mov rax, 3
  ret
```

コンパイラの最初の仕事は、まさにこの程度の対応関係を作ることです。入力が整数 1 個だけなら、対応する `mov` と `ret` を出力すれば十分です。ここで「高級言語の式も結局は命令列へ落ちるのだ」という感覚を掴めれば十分です。

簡単な式の例

式が `1+2` になると、左右の値をどこかへ積んでおき、最後に足し合わせる必要があります。スタックを使う方法なら、左辺を評価して `push` し、右辺を評価して `pop` した値と演算するだけで実装できます。

```
mov rax, 1
push rax
mov rax, 2
pop rdi
add rax, rdi
```

この方法は効率はよくありませんが、説明用としては非常に明快です。後でレジスタ割り当てや最適化を入れるとしても、まずは「式木を深さ優先にたどりながらスタックマシン風にコードを出す」という単純なモデルで十分です。

関数呼び出しを含む例

関数呼び出しまで進むと、呼び出し規約が重要になります。x86-64 System V ABI では、最初の数個の整数引数は `rdi`、`rsi`、`rdx`、`rcx`、`r8`、`r9` といったレジスタで渡され、戻り値は `rax` に置かれます。

つまり `foo(1, 2)` を呼ぶには、単に `call foo` を出すだけでは足りません。呼び出し前に引数を正しい場所へ並べ、必要ならスタックアラインメントを保ち、呼び出し後に結果を `rax` から受け取る必要があります。

```
mov rdi, 1
mov rsi, 2
call foo
```

コンパイラが大きくなるにつれて、このような「言語仕様ではなく ABI に由来する制約」が増えていきます。関数が増えたとたん急に難しく感じるのは、この外部との約束事が一気に増えるからです。

アセンブリを読む練習

アセンブリへ慣れる最も良い方法は、短い C コードをコンパイルして結果を読むことです。最初は次のようなプログラムで十分です。

```
int add1(int x) {
  return x + 1;
}
```

これを読むときは、細部の命令より「引数 x はどこから来るか」「+1 はどこで行われるか」「戻り値はどこへ置かれるか」という観点で追うと理解しやすくなります。コンパイラ実装でもまったく同じ視点を使います。

同じように、`if` や `while` を含む短いコードも眺めてみると、ラベルとジャンプがどのように現れるかが見えてきます。自分が後で生成するコードを先に読む経験は、コード生成器を書くときの大きな助けになります。

デバッグで見るべき点

生成したアセンブリが正しく動かないとき、次の観点で見ると原因を絞りやすくなります。

- 一時値を積んだスタックが最後に元へ戻っているか
- 関数呼び出し前後で必要なレジスタやスタック整列が崩れていないか
- 比較命令の直後に正しい条件ジャンプを出しているか
- ローカル変数のアドレス計算に使うオフセットがずれていないか

前半の小さなコンパイラでは、こうした確認を人間の目で十分追えます。だからこそ、最初からアセンブリ出力を選ぶ価値があります。生成物が黒箱でないことは、教材として非常に重要です。

命令を読むときの観点

アセンブリを見るときは、細かい命令名を全部覚えようとするより、次の観点で読むと整理しやすくなります。

- 値は今どこにあるか
- 次の命令はどの値を消費し、どこへ結果を置くか
- 条件分岐の判断材料はどこにあるか
- この関数の入口と出口でスタックはどう変化するか

この見方ができるようになると、生成されたアセンブリを眺めて「ここでローカル変数のアドレスを計算している」「ここで条件が偽なら else 節へ飛んでいる」といった意味が読めるようになります。コンパイラのデバッグ能力は、そのままアセンブリ読解力に直結します。

本章のまとめ

ここで覚えておきたいのは、CPU はレジスタとメモリを相手に命令を実行し、アセンブラは人間向け表記を機械語へ変換する、という 2 点です。コンパイラはその前段に位置し、高級言語の意味を「CPU が実行可能な命令列」に落とし込む翻訳器だと考えると見通しがよくなります。

次章からはいよいよ、整数式だけを扱う極小の言語を実際にコンパイルしていきます。最初は非常に小さな範囲ですが、この小ささこそが重要です。ここでコード生成の最初の成功体験を作ると、以後の機能追加がぐっと楽になります。

電卓レベルの言語の作成

最初の目標は、四則演算ができるだけの極小言語をコンパイルすることです。ここで重要なのは、機能の小ささではありません。入力を字句解析し、構文解析し、構文木へ変換し、アセンブリを出力するという一連の流れを、最小構成で最後まで通すことが重要です。

この章で作るものは、一見すると「少し賢い電卓」にしか見えません。しかし実際には、後に巨大化していくコンパイラのほぼすべての骨格が、ここにすでに登場します。字句解析、パーサ、構文木、エラー報告、コード生成、テスト。これらを小さな範囲で確実に経験しておくことが、後半の難しさを大きく下げます。

ステップ 1: 整数 1 個をコンパイルする

最初の版では、入力として 1 個の整数しか受け付けません。たとえば 42 を読んだら、戻り値として 42 を返す関数のアセンブリを出力します。言語らしさはほとんどありませんが、パイプライン全体を通す最初の成功としては十分です。

```
入力: 42
出力イメージ:
  mov rax, 42
  ret
```

この段階で重要なのは、コンパイラ本体よりむしろ周辺の流れです。コマンドライン引数から入力を受け取り、整数として解釈し、アセンブリを標準出力へ書き出し、その結果をアセンブラとリンカへ渡して実行する。ここで最初の「ソースから実行へ」のパイプラインが完成します。

実装のポイント

実装は素朴で構いません。入力文字列を `strtol` のような関数で整数へ変換し、その値を埋め込んだアセンブリ文字列を出力するだけです。まだトークナイザもパーサもありません。それでも「入力を別の表現へ翻訳する」という意味で、すでにコンパイラです。

このとき、出力するアセンブリ全体の雛形も用意しておくと後が楽になります。関数ラベル、`mov`、`ret` という最小構成でよいので、「コンパイラが何を吐くのか」を毎回一定の形式にしておくと、以後の変更が差分として読みやすくなります。

このステップで見える課題

もちろん、この方法は少し式が複雑になっただけで破綻します。1+2 を見た瞬間に、整数 1 個だけを読む実装では足りないことがわかります。つまり、このステップの役割は「パイプラインの最小成功」を作ることであり、言語仕様を広げることではありません。

小さすぎるように見える段階でも、ここで得られるものは大きいです。コンパイル結果を実際に走らせるところまでできれば、以後の作業は「より多くの入力が通るようにする」問題へ整理されます。これが最初の心理的な壁を大きく下げます。

ステップ 2: + と - を実装する

次は加算と減算です。`1+2-3` のような式を左から順に読めるようにします。ここで初めて、入力をただ整数として読むだけでは足りず、演算子とオペランドを区別する必要が出てきます。

最も単純な方法は、数字を読み取ってはその場で足し引きすることです。まだ厳密なパーサを導入しなくても、左から順に走査して `+` と `-` に応じて結果を更新するだけで動きます。教材としては、まずはこの粗い形でも構いません。

なぜこの段階を挟むのか

いきなり本格的な構文解析へ進むこともできますが、この小さな中間段階には意味があります。入力を 1 文字ずつ読みながら「ここで何を見ているか」を意識する練習になるからです。数字列と記号列の区別、空白の扱い、入力末尾の検出など、字句解析の萌芽がここにあります。

また、コード生成も少しだけ複雑になります。最終結果が定まったあとに `mov rax, result` としてもよいですが、将来は演算の構造を保持したままアセンブリを出したくなります。この違いに気づくこと自体が、次のトークナイザ導入への準備になります。

よくある雑な実装とその限界

この段階では、`isdigit` で数字を読み、`+` と `-` を見たら足し引きするだけのコードになりがちです。それ自体は悪くありません。しかし、`12+34` のように桁数が増えたり、空白が入ったり、誤った入力が来たりすると、すぐに境界条件が増えます。

この「少し仕様を増やしただけでコードが急に汚れる」という経験は重要です。コンパイラでは早い段階で責務分割が必要になることを、体験として知るからです。次のステップでは、その問題を字句解析器の導入で解消します。

ステップ 3: トークナイザを導入する

字句解析器、つまりトークナイザを入れると、構文解析器は文字列の細部を気にせず、「次は整数トークンか、記号トークンか」という単位で入力を読めるようになります。ここでコンパイラの内部構造が一段整理されます。

トークナイザの責務は、空白を飛ばし、数字列を 1 個の整数トークンへまとめ、記号を対応する種類へ変換することです。後から予約語や複数文字演算子が増えても、パーサが文字列処理に汚染されずに済みます。

トークン列という考え方

トークン列は、ソース文字列から余計な表記上の細部を取り除いた表現だと考えるとわかりやすいです。たとえば `12 + 34` と `12+34` は、人間には同じ式に見えます。トークナイザが空白を消し、整数と `+` を切り出してしまえば、構文解析器にとって両者はまったく同じ入力になります。

```
ソース: 12 + 34
トークン列:
  NUM(12), '+', NUM(34), EOF
```

この正規化は、その後の処理を劇的に簡単にします。パーサはもう「今見ている文字が空白かどうか」を気にしなくて済みますし、コード生成器は文字列ではなく構造化された入力だけを相手にできます。

トークナイザのデータ構造

最初は配列でも連結リストでも構いません。必要なのは、トークンの種類、必要なら数値、元の文字列位置くらいです。後でエラー報告を改善するとき、元の位置情報を持っている価値が効いてきます。

ここで「元の文字列へのポインタ」と「長さ」を保持する設計にしておくと、後に複数文字識別子や予約語を扱う際にもそのまま使い回せます。トークンの値を毎回新しく文字列コピーしなくてよいので、実装もシンプルになります。

パーサが楽になる

トークナイザ導入後のパーサは、`consume("+")` や `expect_number()` のような補助関数を使ってトークン列を読む形になります。これにより、構文規則そのものに集中できるようになります。トークン化の恩恵は、機能追加のたびに大きくなります。

ステップ 4: エラー報告を改善する

教材実装では後回しにされがちですが、エラー報告は早めに整えたほうが開発効率が上がります。失敗した位置を示し、「ここでは整数が必要だが + が来た」と言えるだけで、実装の試行錯誤がかなり楽になります。

コンパイラは、正しいプログラムを通すだけでなく、間違ったプログラムをどう拒否するかも重要です。実用上は、後者の品質が低いと開発がひどく苦痛になります。エラー報告の改善は単なる親切機能ではなく、コンパイラ開発の速度そのものに効きます。

最低限ほしい診断

前半の段階で最低限ほしいのは、次の 3 点です。

- どの位置で失敗したか
- 何を期待していたか
- 実際には何が来たか

これだけでも、`1 + * 2` のような入力に対して「`*` の位置で整数が必要」と言えます。字句解析器が元の文字列位置を持っていれば、キャレット付きで位置を表示することも難しくありません。

```
1 + * 2
    ^ ここでは数値か "(" が必要
```

エラー処理は後から足すと辛い

この段階でエラー処理を少し整理しておくと、後々の実装がずっと楽になります。逆に、最後の最後まで `fprintf(stderr, "error\n")` だけで進めると、機能が増えてから診断を改善するのが非常に辛くなります。早めに入口を整えておく価値は大きいです。

文法を導入する

ここから先は、式の構造を文法で表したほうが見通しがよくなります。典型的には次のような形です。

```
expr    = mul (("+" | "-") mul)*
mul     = primary (("*" | "/") primary)*
primary = num | "(" expr ")"
```

この形にしておくと、掛け算が足し算より強く結びつくことが自然に表現できます。再帰下降構文解析では、この文法に対応する `expr()`、`mul()`、`primary()` を書けば、そのままパーサになります。

文法を入れる意味

文法を導入する本当の価値は、優先順位だけではありません。コードの責務分割が自然に決まることです。`expr()` は加減算、`mul()` は乗除算、`primary()` は数値や括弧式というように、それぞれが明確な役割を持つようになります。

この構造があると、新しい演算子を追加する場合にも、どの層へ入れるべきかがすぐにわかります。比較演算子を足すなら `expr` より弱い層、単項演算子を足すなら `primary` より強い層、といった設計判断がしやすくなります。

左再帰に注意する

再帰下降構文解析では、左再帰な文法をそのまま使えません。たとえば `expr = expr "+" mul | mul` のような書き方は無限再帰になります。そのため、前述のように繰り返し構文へ変形した形を使います。ここで理論と実装が接続します。

構文木を作る

構文解析の次に行うのは構文木の構築です。`1+2*3` なら、「根が `+`、左が `1`、右が `*`」という木を作ります。これにより、コード生成は入力文字列ではなく木構造を相手にできるようになります。

```
    (+)
   /   \
 (1)   (*)
      /   \
    (2)   (3)
```

なぜ木が必要なのか

入力文字列を左から読むだけでも、その場で評価することはできます。しかしコンパイラでは、後の処理で式の構造を何度も参照したくなります。たとえばコード生成では、左右の部分式を別々に評価し、結果を組み合わせる必要があります。型付けでは、「この `+` の左右は何型か」を知りたくなります。木があれば、そうした処理を自然に書けます。

ノードの種類

最初の段階で必要なノードは多くありません。整数リテラル、二項演算子、必要なら単項演算子程度で十分です。それでも、ノード種別と左右の子、必要なら値を持つ構造体を `1` つ定義しておくと、以後の拡張が楽になります。

スタックマシンとしてコード生成する

最初のコード生成は、式木を深さ優先でたどりながら値をスタックへ積む方法がわかりやすいです。左部分木を評価して `push`、右部分木を評価して `pop` した値と演算し、結果を再び積む、という流れです。

たとえば `1+2` のような式は次のように出力できます。

```
mov rax, 1
push rax
mov rax, 2
pop rdi
add rax, rdi
```

この方法は、レジスタ利用の効率は高くありません。しかし、式木の形がそのままコード生成の再帰構造に対応するため、教材としては非常に優れています。「左右を先に計算してから演算する」という高級言語の構造が、ほぼそのままアセンブリへ落ちていくのが見えやすいからです。

一時値をどこへ置くか

コード生成で最初に迷うのは、一時値の置き場所です。レジスタを賢く割り当てる方法もありますが、前半ではスタックを一時退避場所として使うほうが素直です。後で最適化やレジスタ割り当てに進むとしても、まずは正しいコードが出ることを優先すべきです。

ステップ 5: * と / を実装する

乗算と除算は、文法を導入した恩恵がもっともよく出るステップです。もし式を単に左から処理していたら、`1+2*3` を正しく扱えません。`mul()` を 1 段用意しておけば、自然に優先順位を実装できます。

優先順位が実装へ現れる瞬間

`expr()` が `mul()` を読む形にしておくと、`1+2*3` を読んだとき、右辺の `2*3` が 1 個の `mul` として先にまとまります。つまり、文法上の階層がそのまま演算子優先順位の実装になっています。この対応を 1 度体験すると、後に比較演算や代入を追加するときにも応用が利きます。

除算で見えてくること

除算自体は、命令としては `idiv` などを使えば済みます。ただしここで、「CPU 命令には少し癖がある」ことも見えてきます。除算の前に特定のレジスタを準備しなければならない、商と余りの置き場所が決まっている、といった事情です。高級言語の演算子が必ずしも単純な 1 命令へ対応しないことを知るよい機会です。

ステップ 6: 単項 + と -

`-3` のような式を扱うには、二項演算子としての `-` と、符号反転としての単項 `-` を区別しなければなりません。ここでも文法が役立ちます。

`unary = ("+" | "-")? primary` のような層を 1 つ挟めば、単項演算子を素直に扱えます。構文木上では `0-x` に落としてもよいですし、単項マイナス専用ノードを作っても構いません。教材の初期段階では、前者のほうが実装を流用しやすいことが多いです。

どこで曖昧さが生まれるか

`-` が現れたとき、それが二項減算なのか単項負号なのかは、前後関係で決まります。入力の文字としては同じでも、構文上の意味は異なります。こうした「見た目は同じだが文脈で意味が変わる」要素は、今後も C のあちこちで登場します。早い段階で慣れておくと後半が楽です。

ステップ 7: 比較演算を追加する

`==`、`!=`、`<`、`<=` を入れると、言語が急にプログラムらしくなります。これらは後に `if` や `while` を実装する際の基礎になります。数値計算だけだった世界から、「条件に応じて振る舞いを変える」世界へ一歩近づきます。

比較演算子はトークナイザにも変更が必要です。<= は < と = の 2 文字列ですが、意味的には 1 個の演算子です。したがって、字句解析器は最長一致で 2 文字演算子を読み取る必要があります。ここで初めて、「トークン化は単なる 1 文字切り出しではない」ことが明確になります。

文法上の位置

構文的には、加減算より弱い優先順位の層として `relational` や `equality` を追加すると整理しやすくなります。たとえば次のような形です。

```
equality   = relational (("==" | "!=") relational)*
relational = add (("<" | "<=" | ">" | ">=") add)*
add        = mul (("+" | "-") mul)*
```

こうして層を 1 段ずつ増やす考え方は、後に代入、論理演算、条件演算子を入れるときにもそのまま使えます。C の式構文が複雑に見えるのは、こうした層が多いからであって、1 層ずつ見れば特別な魔法ではありません。

コード生成の変化

コード生成では、`cmp` と条件分岐、あるいは `sete` / `setne` / `setl` のような命令を用いて、真偽値を 0 か 1 に落とし込みます。ここで初めて CPU フラグを本格的に使うことになります。高級言語の比較演算が、低レイヤでは「比較してフラグを立て、それを数値化する」処理だと実感できる場面です。

ここまでのテストケース

この段階まで来たら、短い式を大量に通しておくと安心です。たとえば次のような入力は、前半の土台が正しくできているかを手早く確認するのに向いています。

```
0
42
1+2
5-3-1
1+2*3
(1+2)*3
-3+5
1==1
1!=2
1<2
2<=2
```

こうしたテストを通しておくと、後に変数や制御構文を入れたときも、式部分だけが壊れていないかをすぐ確認できます。コンパイラは機能追加のたびに既存機能を壊しやすいので、早い段階の小さなテスト群が後半まで効きます。

前半の設計で残してよい粗さ

この時点では、まだ粗くてよい部分も多くあります。たとえば最適なレジスタ利用、洗練されたエラーメッセージ、完全な整数演算の扱いなどは後回しで構いません。重要なのは、トークナイザ、パーサ、構文木、コード生成という責務分割ができていることです。

逆に、この段階でごまかさないほうがよいのは、文法の階層と構文木の設計です。ここが曖昧なまま進むと、後の制御構文や代入演算子の追加で一気に苦しくなります。前半では「あとから差し替えやすい粗さ」と「今のうちに整えておくべき芯」を見極めるのが大切です。

この段階で見えてくること

たった数ステップでも、コンパイラの主要部品が一通り揃ったことがわかります。入力の字句解析、文法に基づく構文解析、構文木、スタックベースのコード生成、エラー報告です。すでに「言語を別の表現へ翻訳する装置」としての形はできています。

さらに重要なのは、ここまでの設計が後半そのまま生きることです。トークナイザは予約語や識別子を読むために拡張され、パーサは文と宣言を扱うようになり、構文木は文ノードや型情報を持つようになり、コード生成はローカル変数や制御構文を扱うようになります。骨格は変わりません。

次章では、この最小コンパイラへ「文」と「関数」を導入して、プログラムらしさを一気に増やしていきます。ここで初めて、式を計算するだけの世界から、状態と制御を持つ世界へ踏み込みます。

分割コンパイルとリンク

コンパイラをある程度書いたところで、生成したアセンブリをどう実行形式へ結びつけるのかを理解しておくと見通しがよくなります。特に複数ファイルや標準ライブラリを扱い始めると、リンカの役割を知らずには進めません。

前章までは「1 つの式から 1 つの関数を作る」くらいの小さな世界でした。しかし現実のプログラムは、複数の関数、複数のファイル、外部ライブラリから成り立っています。その全体を 1 回のコンパイルで全部処理するのではなく、部品ごとに処理し、最後に結合する仕組みが必要になります。それがオブジェクトファイルとリンクです。

オブジェクトファイルとは何か

コンパイラやアセンブラが吐く中間成果物がオブジェクトファイルです。ここには機械語だけでなく、未解決のシンボル参照や再配置情報が含まれます。つまり「この位置には後で `printf` のアドレスを埋め込んでほしい」という情報も一緒に持っています。

オブジェクトファイルは、まだ完成した実行形式ではありません。いわば「部品がある程度できた状態」のファイルです。コードもデータも入っていますが、他のファイルとの結合が終わっていないため、最終アドレスは未定です。この未定な部分を、リンカが最後に埋めます。

再配置情報の意味

たとえば `call foo` という命令があるとして、その時点で `foo` の実アドレスがまだ決まっていないことがあります。その場合、オブジェクトファイルには「この位置は `foo` を参照しているので、あとで正しい値へ書き換えてほしい」という印が残ります。これが再配置情報です。

最初の教材実装では、この情報を自力で扱う必要はありません。しかし、リンカが何をしているのかを理解するには非常に重要な概念です。コンパイラが外部関数を呼べるのは、最後に誰かがこの空白を埋めてくれるからです。

ライブラリと再利用

分割コンパイルの利点は、変更のあったファイルだけを再コンパイルすればよいことです。大きなプロジェクトで全ソースを毎回最初からコンパイルするのは非効率です。オブジェクトファイルを単位に分けることで、ビルド時間と依存関係の管理が現実的になります。

本書のコンパイラも、将来的にはランタイム補助関数やテストコードと組み合わせて使うことになります。そのとき、リンカが「あとで埋める」仕組みを持っていることを知っている

だけで設計がずっと楽になります。自分のコンパイラは「正しいシンボル参照を出力する」までを担当し、結合はツールチェインへ委ねればよいからです。

静的ライブラリと共有ライブラリ

この段階では詳しく踏み込みませんが、ライブラリには静的に取り込むものと、実行時に読み込むものがあります。どちらの場合でも、コンパイラは最終アドレスを知っている必要はありません。必要なのは、名前と呼び出し規約の約束を守ることです。後に動的リンクの章へ進むと、この視点が活きます。

関数とローカル変数

ここからは、式だけでなく文や関数を扱う本格的なコンパイラへ進みます。プログラムらしさを増やすために、まずはブロック、制御構文、ローカル変数、関数定義を順に導入します。前章までで作った式コンパイラは、そのまま「文の中で使われる式」として再利用できます。

ステップ 8: 文を読む

最初に必要なのは、式だけでなく文の列を読めるようにすることです。セミコロンで終わる式文、空文、ブロック文を扱えるようにすると、関数本体を 1 本の木として表現できるようになります。

構文木のノード種類も増えます。式ノードだけでなく、`return`、`if`、`while`、`for` のような文ノードが必要になり、コード生成側も「値を計算する」のではなく「制御をどこへ流すか」を考える場面が出てきます。

文と式の違い

式は値を持ちますが、文は必ずしも値を持ちません。`1+2` は値 3 を持つ式ですが、`if (...) ...` は値を返さない文です。この区別を内部表現へ持ち込むことで、パーサもコード生成器も責務が整理されます。

とはいえ、C では式文のように「式が文になる」ケースもあります。したがって、完全に別世界として切り離すのではなく、「文ノードの一種として式文を持つ」と考えると自然です。

ステップ 9: 1 文字のローカル変数

変数が入ると、コンパイラは名前と記憶場所の対応を管理しなければならなくなります。最初は 1 文字の識別子だけでも構いません。`a` や `b` のような変数名をローカル変数表へ登録し、スタックフレーム上のオフセットへ割り当てます。

ここで一気に「記号の意味」が増えます。今までは `1` や `+` のような演算の材料しかありませんでしたが、`a` は「ある場所に格納された値」を指す名前になります。名前解決が必要になる最初の段階です。

左辺値という概念

ローカル変数を実装すると、左辺値という概念が必要になります。`a=3` の左辺は値そのものではなく、書き込み先のアドレスを表します。これ以後、式を評価する関数と、左辺値のアドレスを求める関数を分けて考えると整理しやすくなります。

この区別は後に `*p = 3` や `x.y = 1` を実装するときにもそのまま使います。つまり、ローカル変数は単に便利な機能であるだけでなく、左辺値という C らしい概念へ入るための入口でもあります。

ステップ 10: `return`

`return` は、関数から脱出して呼び出し元へ戻る文です。制御フローが線形でなくなる最初の例でもあります。実装としては、式を評価して戻り値レジスタへ入れ、エピローグへジャンプする形がわかりやすいです。

なぜ共通エピローグにするのか

関数の出口を 1 箇所に集めておくと、ローカル変数の破棄やスタックポインタの復元が簡単になります。この「共通エピローグ」方式は後の `break` や `goto` を考えるときにも応用できます。複数の `return` 文があっても、最後に片付ける場所を 1 つ持っておけば整然としたコードになります。

戻り値レジスタの存在

ここで、戻り値は ABI によってどのレジスタへ返すか決まっている、という事実も実感できます。言語仕様だけではなく、外部世界との約束がコード生成に直接現れる瞬間です。コンパイラは単独で閉じた世界のプログラムではないことが、ますますはっきりします。

ステップ 11: 複数文字のローカル変数

1 文字の変数名ではすぐに不便になるので、トークナイザを拡張して複数文字の識別子を読めるようにします。英字、数字、アンダースコアの組み合わせを 1 個のトークンとして扱い、予約語と通常の識別子を区別します。

この変更は見た目以上に大事です。ここから先、変数だけでなく関数名、型名、構造体メンバ名など、識別子がコンパイラの世界を埋めていきます。したがって、識別子の扱いを早めにきちんと整えておく必要があります。

予約語との区別

ここで初めて「予約語か、ただの識別子か」という問題が明確になります。`return` や `if` は予約語なので変数名としては使えません。この区別は、字句解析時に行ってもよいですし、識別子として読んだあとに予約語表で判定しても構いません。

後の `typedef` 実装では、識別子が型名か変数名かも区別する必要が出ます。したがって、「文字列から意味を引く」仕組みを少しずつ育てていくことになります。

ステップ 12: 制御構文

`if`、`while`、`for` を追加すると、プログラムは単なる式の集まりではなくアルゴリズムを記述する言語になります。コード生成ではラベルとジャンプ命令が主役になります。

たとえば `if (cond) then else` なら、条件を評価し、偽なら `else` ラベルへ飛び、真なら then 節を実行し、最後に end ラベルへ飛ぶ、という形になります。`while` や `for` も同じで、ラベルをいくつか生成して制御を結びます。

制御構文は「文脈」を持つ

ここで重要なのは、コード生成が式単位から文脈依存へ変わることです。`if` の中で生成すべきラベルは、その `if` 専用でなければなりません。`while` の `continue` 先は、別のループとは異なる場所になります。つまり、コード生成は単なる再帰ではなく、「今どの構文の中にいるか」という文脈を持ち始めます。

`for` 文が持つ複数の場所

`for` 文には、初期化、条件、増分、本体の 4 要素があります。コード生成では、それぞれが別の位置に現れます。特に `continue` を入れると、増分式の直前へ飛ぶ必要があるため、ラベ

ルの置き方が `while` と微妙に異なります。最初は `for` を `while` へ展開するイメージを持つと整理しやすいです。

ステップ 13: ブロック

文が増えると、複数文を 1 つに束ねるブロックが必要になります。`{ ... }` を 1 個の文として扱えるようにすると、`if` の本体や関数本体を素直に表現できます。

ブロックは後にスコープの境界としても重要になります。今は文の列を持つだけで十分ですが、将来 `typedef` やローカル変数のスコープを扱う段階で同じ構造が活きてきます。つまり、ここで入れるブロック構造は後の意味解析への先行投資です。

ステップ 14: 関数呼び出し

関数を呼び出せるようになると、標準ライブラリやテスト補助関数を再利用できます。実装の要点は、引数を呼び出し規約に従って並べることと、呼び出し後の戻り値を `rax` から受け取ることです。

この時点で、関数名をトークンとして読み、引数リストをパースし、呼び出しノードを構文木へ追加します。引数の数に応じたレジスタ割り当てが必要ですが、最初は整数引数だけに限れば十分です。

スタックアラインメントの注意

関数呼び出しでは、スタックが特定の境界に揃っていることを ABI が要求する場合があります。教材の最初の段階では軽視しがちですが、外部関数を正しく呼ぶには重要です。たまたま動いたコードが、最適化や別環境で壊れる原因になりやすいからです。

評価順序の感覚

引数を複数持つ関数呼び出しでは、各引数の式をどう評価するかも設計に関わります。最初は単純に左から構文木をたどってスタックへ積み、最後にレジスタへ並べ直す方法で十分です。効率はよくなくても、実装はずっと見通しがよくなります。

ステップ 15: 関数定義

最後に関数定義を読めるようにすると、単一の式コンパイラが本当の意味で C コンパイラらしくなります。関数ごとにローカル変数表とスタックサイズを計算し、プロローグとエピローグを生成します。

パーサの入口も、もはや「式を 1 個読む」ではなく「トップレベルの関数定義を繰り返し読む」へ変わります。ここまで来ると、自分のコンパイラ自身を少しずつ C で書き始める準備が整ってきます。

関数ごとの独立性

関数定義を入れると、ローカル変数表やラベル生成の状態を関数ごとにリセットしなければなりません。これは「スコープ」と「翻訳単位」の感覚に直結します。どこまでが 1 つの独立したコンテキストかをきちんと切ることが、後のグローバル変数や外部宣言にもつながります。

x86-64 の呼び出し規約

関数を実装するときには ABI が避けて通れません。引数をどのレジスタへ置くか、戻り値はどこへ返すか、スタックを 16 バイト境界に揃える必要があるか、といった規則は言語仕様ではなくプラットフォーム仕様です。

この章では細部まで暗記する必要はありませんが、コンパイラが単独で完結せず、外部のツールや実行環境と約束事を共有しているという点は強く意識しておくべきです。ここを曖昧にしたまま進むと、「自分で生成した関数同士は動くが、外部関数を呼ぶと壊れる」という種類の不具合に悩まされます。

コンピュータにおける整数の表現

整数はソースコード上では単純に見えても、実際の機械では固定長のビット列です。その事実を理解しておくと、比較、符号拡張、キャスト、ビット演算の挙動が格段にわかりやすくなります。前半のうちは整数を抽象的な数学的対象として扱っていても進めますが、後半に向かうほどビット列としての実体が効いてきます。

符号なし整数

符号なし整数は、ビット列をそのまま自然数として読むだけです。8 ビットなら 0 から 255 まで、32 ビットなら 0 から 2^32-1 までを表せます。加算や減算で範囲を超えると下位ビットだけが残ります。

この性質は、コンパイラが特別な魔法をしているわけではなく、CPU が固定長の器で計算していることに由来します。高級言語の整数計算も、最終的にはこの器の中で行われます。

符号付き整数と 2 の補数

現代的な CPU では、符号付き整数はほぼ例外なく 2 の補数で表されます。負数をビット反転して 1 足す、という説明で知られている表現です。これにより、加算器を符号付き・符号なしで共通化できるため、ハードウェア実装が単純になります。

たとえば 8 ビットで `-1` は `11111111`、`-2` は `11111110` のように表されます。正の数と負の数が同じ加算器で扱えるのは、2 の補数表現の大きな利点です。

符号拡張

小さい幅の整数を大きい幅へ広げるとき、符号なし整数は 0 で埋め、符号付き整数は最上位ビットを複製して埋めます。これが符号拡張です。`char` や `short` を `int` へ持ち上げる場面で頻繁に出てきます。

この違いを理解していないと、`char` の比較や代入で不思議な振る舞いに見えることがあります。コンパイラがロード命令を選ぶときにも、符号ありとして読むのか、ゼロ拡張するのかが重要になります。

否定とオーバーフロー

`-x` は、0 から `x` を引くことと同じです。2 の補数表現ではビット反転して 1 足す実装とも対応します。ただし最小値を反転すると表現範囲を超えるため、言語仕様上やハードウェア上の細かな注意が必要です。

教材の前半ではそこまで深く気にしなくても進めますが、整数の実体がビット列だという感覚は、後の型変換や比較命令の選択で必ず役立ちます。高級言語の整数が「無限精度の数学的整数」ではないことを意識するだけで、コード生成の選択理由が見えやすくなります。

制御構文のコード生成を読む例

`if` や `while` のコード生成は、最初は長く見えても、実際にはラベルとジャンプの組み合わせです。たとえば次のようなコードを考えます。

```
int main() {
  int x;
  x = 0;
  while (x < 3)
    x = x + 1;
  return x;
}
```

このときコード生成器は、おおむね「ループ先頭ラベル」「条件評価」「偽なら脱出」「本体」「先頭へ戻る」「脱出ラベル」という形で命令列を作ります。高級言語の反復構文も、低レイヤではジャンプの組み合わせだと意識できると、後の break や continue が理解しやすくなります。

8 ビット整数で考える練習

2 の補数や符号拡張が直感しにくいときは、8 ビット世界で手計算してみると理解しやすいです。たとえば 11111111 が符号付きでは -1、符号なしでは 255 に見えることや、11111111 を 32 ビットへ符号拡張すると上位も 1 で埋まることを手で確認すると、char や int の変換がかなり具体的になります。

この感覚は、後に char 型やキャストを扱うときに直接効きます。整数表現の話は抽象理論ではなく、後半の実装判断の基礎です。

この章の位置づけ

この章で扱った関数、ローカル変数、制御構文、整数表現は、以後のすべての土台になります。関数がなければプログラムは組み立てにくく、ローカル変数がなければ状態を持てず、制御構文がなければ分岐や反復が書けません。さらに、整数表現の理解がなければ、後の小さい型やキャストの説明が浮いたものになります。

次章では、ここへポインタ、配列、sizeof、グローバル変数、文字列リテラルを加え、C らしさをさらに強くしていきます。ここから先は、型を意識しない設計が通用しなくなる場面が増えてきます。

ポインタと文字列リテラル

ここから先で、コンパイラは一気に C らしくなります。ポインタ、配列、char、グローバル変数、文字列リテラルが入ると、扱えるプログラムの幅が大きく広がります。同時に、型を意識しない設計ではすぐに無理が出ることも見えてきます。

前章までは、まだ「整数式を評価する少し賢い言語」の延長で進められました。しかしポインタが入ると、値が何バイト幅なのか、アドレスと値はどう違うのか、左辺値はどこで必要になるのか、といった問題が急に前景化します。C を C らしくしている核心のかなり大きな部分が、この章にあります。

ステップ 16: 単項 & と *

ポインタを導入する最初の一歩は、アドレスを取る & と、ポインタを参照する * を入れることです。&x は x のアドレス、*p は p が指す先の値を表します。

この 2 つは対になる演算子ですが、意味はかなり異なります。& は左辺値のアドレスを求める演算であり、* はポインタ値を左辺値へ変換する演算です。表面的には単項演算子が増えただけに見えても、内部では「値」と「場所」の区別がより重要になります。

& の意味

&x が返すのは、x という変数に格納された値ではなく、その変数が置かれているアドレスです。したがって & は右辺値を作る演算ではあるものの、その入力として左辺値を必要とします。単なる数値定数に & を付けられないのはそのためです。

* の意味

一方の *p は、「ポインタが指している先の場所」を意味します。この表現が代入の左辺にも右辺にも現れることが、C の面白いところです。x = *p では右辺値として使われ、*p = 3 では左辺値として使われます。したがって、* の実装では「アドレスをたどって値を読む」と「書き込み先として場所を返す」の両面を意識する必要があります。

左辺値を扱う内部関数

この時点で、左辺値を扱う内部関数をきちんと持っているかどうかが実装を左右します。変数名を読んだらその値を返すのではなく、まず場所を表す概念として扱い、必要に応じてロードするという設計が後々効いてきます。後の配列、構造体、複合代入でもこの設計が再利用されます。

ステップ 17: int キーワード

変数や関数を本格的に宣言し始めると、型名を明示する必要が出てきます。最初は int だけでも十分です。int x; や int foo(); のような宣言を受け付けられるようにすると、以後の型拡張が自然になります。

ここでのポイントは、構文解析の入口が「式」だけでなく「宣言」も扱うようになることです。C のトップレベルやブロック先頭では、式が来るのか宣言が来るのかを見分ける必要があります。前半までは「式を読む関数」が入口でしたが、ここからは文脈によって構文の分岐が増えます。

型名が文法へ入る

int を入れることで、初めて「型名も文法の要素である」という事実が明確になります。後で char、ポインタ、配列、struct、typedef が増えると、この流れはさらに強まります。つまり int は単なるキーワード追加ではなく、C の宣言構文へ入るための入口です。

ステップ 18: ポインタ型

int *p; のような宣言を読むには、型データ構造に「基底型へのポインタ」という概念を入れなければなりません。* は単項演算子としても宣言子の一部としても出てくるので、文脈に応じて意味が変わります。

型データ構造が必要になる理由

前半では、式の値はほとんどすべて整数だとみなして進められました。しかしポインタが入ると、それではもう足りません。x+1 と p+1 は見た目は同じでも意味が違い、*p が合法かどうかは p の型を知らなければ判断できません。

したがって、型は単なる注釈ではなく、コード生成そのものに影響します。ここで型データ構造をきちんと導入しておくと、後に sizeof、配列、struct へ進むときにも一貫した設計を保てます。

型の大きさ

型を導入すると、各型が何バイトかという情報を持ちたくなります。`int` は 4 バイト、`char` は 1 バイト、ポインタは 64 ビット環境なら 8 バイト、といった具合です。後に `sizeof` やポインタ加算を実装するとき、サイズ情報は不可欠です。

このサイズ情報は、単に `sizeof` を返すためだけのものではありません。たとえばスタック上にローカル変数領域を確保するとき、各変数のサイズがわからないとオフセットが計算できません。型のサイズは、配置とコード生成の両方に関わります。

左辺値と右辺値の整理

ポインタが入ると、左辺値の意味がより重要になります。`*p = 3` の左辺は、単なる変数ではなく「ポインタをたどった先の記憶場所」です。式が「値」を返すだけでなく、「書き込み可能な場所」を返すこともあるという視点を、ここでしっかり持っておく必要があります。

ステップ 19: ポインタ加算

`p+1` は整数加算とは異なります。`p` が `int *` なら 4 バイト進み、`char *` なら 1 バイト進みます。つまり、ポインタ演算は基底型のサイズに応じたスケーリングを伴います。

なぜスケーリングが必要なのか

ポインタは「次の要素」へ進みたいのであって、「次の 1 バイト」へ進みたいわけではありません。`int *p` に 1 を足すとき、欲しいのは次の `int` の先頭アドレスです。したがって基底型サイズを掛ける必要があります。ここで、型情報がコード生成に直接現れます。

```
int *p;
p = p + 1;
```

この式の `+ 1` は、実際には「`sizeof(int)` バイトだけアドレスを進める」処理です。整数の `+` と見た目が同じでも、意味はまったく異なります。

引き算や比較への布石

最初は足し算だけを実装すれば十分ですが、後にポインタ同士の差分や比較を考えると、ここで「ポインタ演算は型付き演算だ」という意識を持っておくことが重要です。C のポインタは、ただの巨大整数ではありません。

ステップ 20: `sizeof`

`sizeof` はコンパイル時に型や式の大きさを調べる演算子です。評価時に実際のメモリアクセスを行わないという点が重要です。`sizeof(*p)` を計算するために `p` をデリファレンスしてはいけません。

`sizeof` が教えてくれること

`sizeof` を入れると、型をきちんと内部表現として持っている価値がさらに増します。単に文字列から推測するのではなく、各ノードに型が付いていれば、サイズを返すだけで済みます。逆に型情報がない設計だと、`sizeof` をきっかけに全体を作り直したくなることがあります。

式と型の両方を取れる

C の `sizeof` は `sizeof(int)` のように型を取る形と、`sizeof x` のように式を取る形の両方があります。この二面性はパーサ実装で少し面倒ですが、後のキャスト構文にも似た問題が出ます。ここで「型名か式かを見分ける」という観点に少し慣れておくと後半が楽になります。

ステップ 21: 配列

配列は、連続したメモリ領域を 1 つの名前で扱うための仕組みです。`int x[3];` のような宣言を読むには、型に「要素型」と「要素数」を持たせます。

型の複雑化

配列が入ると、型は単なる列挙値では足りません。`int`、`int *`、`int[3]` のような再帰的な構造を表現できるデータ構造が必要になります。後の `struct` や関数ポインタにもつながる重要な基礎です。

配列の配置

ローカル配列ならスタック上に連続した領域を確保し、グローバル配列ならデータ領域へ連続配置します。ここで 1 個の値ではなく複数要素の塊を扱うため、オフセット計算の意味がより明確になります。`x[2]` を読むには、基底アドレスから `2 * sizeof(element)` を足した場所へアクセスすればよいわけです。

配列とポインタ

C では、配列は多くの場面で先頭要素へのポインタへ暗黙変換されます。ただし完全に同じものではありません。`sizeof` を取ると違いが現れますし、宣言時の意味も異なります。この「似ているが同一ではない」関係は、C を理解する上で非常に重要です。

ここで曖昧な理解のまま進むと、「なぜ `sizeof(arr)` は大きいのに、関数引数へ渡すとポインタになるのか」が不思議に見え続けます。配列は式脈絡でしばしばポインタへ崩れるが、型としては独立の存在である、という整理をここで持っておくべきです。

ステップ 22: 添字演算子 `[]`

`a[i]` は本質的には `*(a+i)` です。したがって、添字演算子を新しい特殊機能として実装する必要はなく、既存のポインタ加算とデリファレンスへ落とし込めば十分です。

表面構文を既存要素へ還元する

このように、表面上は別機能に見えても、内部表現では既存要素の組み合わせで説明できることが多くあります。コンパイラ設計では、なるべく少ない核となる概念へ還元するのが重要です。そうしておくと、新しい構文が増えても内部の複雑さが爆発しにくくなります。

`i[a]` が合法な理由

C では `a[i]` が `*(a+i)` である以上、理屈の上では `i[a]` も同じ意味になります。普段は使いませんが、この奇妙さは「添字演算子が実は糖衣構文にすぎない」ことを強く印象づけます。教材としては、内部表現のシンプルさを実感するよい例です。

ステップ 23: グローバル変数

ローカル変数がスタック上に置かれるのに対し、グローバル変数はデータセグメントへ置かれます。トップレベルで宣言を読み、シンボルとして名前と型と初期値を保持する必要があります。

トップレベルの意味が増える

グローバル変数が入ると、関数の外にも意味のある構文が現れます。パーサは「これは関数定義か、単なる宣言か」を見分ける必要があり、コード生成もテキストセクションとデータセクションを使い分けるようになります。

スコープ感覚の変化

ここで初めて、ローカルスコープとグローバルスコープの違いがコンパイラへ現れます。名前解決、寿命、配置場所がすべて異なります。後に `extern` や静的記憶域期間の話へ進む前段としても重要です。

ステップ 24: char

`char` を追加すると、文字単位の処理と文字列リテラルが扱いやすくなります。サイズは 1 バイトで、`int` より小さいため、ロードやストアの命令選択でも違いが出てきます。

小さい整数型が持ち込む問題

小さい整数型が入ると、符号拡張やゼロ拡張の話も実用上重要になります。`char` を `int` と同じ感覚で扱うと、比較や代入で思わぬ差が出ることがあります。コンパイラは、ロード時にどのように上位ビットを埋めるかを意識する必要があります。

なぜ char が早めに必要なのか

文字列リテラルやテキスト処理に進むためには、1 バイト単位の型が必要です。したがって `char` は、単に C の型を増やすという以上に、「実用的な入力処理やライブラリ呼び出しに近づく」ための重要なステップです。

ステップ 25: 文字列リテラル

文字列リテラルは、実体としてはヌル終端の `char` 配列です。`"abc"` は、データセクションへ `a`、`b`、`c`、`\0` を並べた静的領域を置き、その先頭アドレスを式の値として返すと考えると実装しやすいです。

コードではなくデータを生成する

このステップで、コンパイラは「入力の一部をそのまま定数データとして実行ファイルへ埋め込む」という仕事を覚えます。コードだけでなくデータも生成物の一部だと実感できる箇所です。前半までの「命令列を出せば終わり」という感覚が少し変わります。

文字列リテラルの一意なラベル

実装上は、各文字列リテラルへユニークなラベルを付け、そのラベルを参照する形でコードを出すと扱いやすいです。複数箇所で同じ文字列が現れたときに共有するかどうかは後で考えてもよく、最初は毎回新しいラベルを振って構いません。

ステップ 26: ファイル入力

ここまでの実装ではコマンドライン引数から短い式を受け取っていたとしても、実用上はソースファイルを丸ごと読みたいはずです。入力をファイルから取り込むように変えるだけで、実際の C コンパイラらしさがかなり増します。

行番号とファイル名

ファイル単位で読むようになると、エラー報告でもファイル名や行番号を持ちたくなります。トークナイザが元の位置情報を保持する設計にしておくと、以後の診断が改善しやすくなります。教材の価値は、通る入力だけでなく失敗時のわかりやすさにも現れます。

入力インターフェースの変化

ここでコンパイラの `main` 関数も少し実用寄りになります。ファイルを開いて全体を読み込み、末尾にヌル文字を足し、トークナイザへ渡す、といった手順が必要になります。この周辺コードは本質ではありませんが、実用上の「使える道具」へ一段近づくステップです。

ステップ 27: コメント

コメントはコンパイル結果へ影響しないので、字句解析で読み飛ばせば十分です。`//` と `/* ... */` の両方を扱えるようにすると、実際の C ソースがかなり通るようになります。

地味だが効く改善

一見地味ですが、コメント対応が入ると既存の C コードをテストへ流用しやすくなります。教材実装から実際のコードへ近づくための重要な一歩です。自分で書いたテストコードにも説明コメントを付けられるようになるので、開発体験も改善します。

トークナイザ責務の確認

ここであらためて、コメント処理がトークナイザの責務だと確認できます。コメントは構文解析の対象ではなく、字句レベルで消してしまうべき表記上の要素です。この役割分担が明確になると、トークナイザとパーサの境界がよりはっきりします。

ステップ 28: テストを C で書き直す

ここまで来たら、テストハーネス自身も C で書けるようになります。アセンブリやシェルだけでなく、自分のコンパイラが生成したコードを、より自然な形で検証できるようになるわけです。

なぜ大きな節目なのか

これは単なるテスト改善ではありません。コンパイラが「自分と同じ言語圏のプログラム」を扱えるようになってきたことの証拠です。セルフホストへの道のりはまだ先ですが、方向としてはかなり近づいています。

テストコードが仕様書になる

コンパイラでは、どの入力を通し、どの入力を拒否するかがそのまま仕様になります。C でテストを書けるようになると、その仕様をより自然な形で蓄積できます。後で機能追加やリファクタをするときにも、安心して動かせる基盤になります。

プログラムの実行イメージと初期化式

ポインタやグローバル変数が入ると、実行ファイルがメモリ上でどう配置されるのかを知っておく意味が増します。同時に、グローバル変数の初期化式が「どこまでコンパイル時に決まっていなければならないのか」も理解したくなります。

実行ファイルの構造

実行形式には、機械語を置くテキストセクション、初期値つきグローバル変数を置くデータセクション、ゼロ初期化領域を表す BSS などがあります。ローダはこれらを適切なメモリアドレスへ配置してからプログラムを開始します。

コンパイラの立場から見ると、「ローカル変数はスタック、グローバル変数はデータ領域、文字列リテラルも静的領域」という整理ができるだけで十分です。どの値がどこに住むかが明確になると、コード生成の分岐も整理しやすくなります。

テキスト、データ、BSS

テキストセクションには機械語が、データセクションには初期値つき静的データが、BSS にはゼロ初期化の静的データが入ります。すべてを 1 種類のメモリとして考えていると、グローバル変数と文字列リテラルの扱いが曖昧になりがちです。セクションの違いを意識すると、生成物全体の見取り図がずっと明確になります。

グローバル変数の初期化式

グローバル変数の初期化は、コンパイル時に確定できる定数式である必要があります。`int x = 3;` や `char *p = "abc";` はよいですが、関数呼び出しの結果で初期化することはできません。

なぜ定数式に限られるのか

グローバル変数は、プログラム開始前にすでにメモリへ配置されている必要があります。したがって、その初期値は実行前に決められていなければなりません。この事情が、ローカル変数との大きな違いを生みます。

実装方針

そのため、初期化式用には通常の式とは少し異なる制約が必要になります。実装としては、初期化子をパースしたあと、その値をデータ列へ直列化する処理を分けて持つと扱いやすくなります。式の意味解析とデータ出力を切り分けて考えるのがコツです。

ローカル変数の初期化

一方、ローカル変数の初期化は実行時に行われます。`int x = y + 1;` のような式も普通に使えます。したがって、ローカル変数の初期化は「代入文へ展開する」ように扱ってしまうのが簡単です。

見た目は似ていても意味は違う

ここでも、見た目は似ていてもグローバル初期化とローカル初期化では意味が違うことがわかります。C は表面上の文法より、どの段階で値が決まるかが重要な言語です。コンパイラを書くと、この違いが非常に具体的に見えてきます。

ポインタと配列の確認例

この章の理解を固めるには、短いコード例を自分のコンパイラで通してみるのが有効です。たとえば次のような断片は、それぞれ別の論点を確認できます。

```
int main() {
  int x;
  int *p;
  p = &x;
  *p = 3;
  return x;
}
```

これは `&` と `*` が両方向に噛み合っているかを確認する例です。次のような例は、配列と添字が正しく処理できているかの確認になります。

```
int main() {
  int x[3];
  x[0] = 1;
  x[1] = 2;
  x[2] = 3;
  return x[0] + x[1] + x[2];
}
```

文字列リテラルや `char` を確認したいなら、短い文字列を返させたり、1 文字を取り出したりする例を作るとよいです。後半で型システムを強化するときも、こうした小さなコード片が回帰テストとして役立ちます。

初期化式でよくあるつまずき

初期化式まわりで詰まりやすいのは、グローバルとローカルの違いを曖昧にしたときです。`int x = 3;` はどちらでも自然に見えるため、実装上の差を見落としがちです。しかしグローバル初期化はデータ生成、ローカル初期化は実行時の代入という違いがあります。

もうひとつのつまずきは、文字列リテラルの実体を「ただのポインタ」と考えすぎることです。実際には静的領域に置かれた `char` 配列であり、その先頭アドレスが式値として現れています。この整理が曖昧だと、配列とポインタの違いも曖昧なまま残ってしまいます。

この章の位置づけ

この章を通して、C の核心にかなり近づきました。ポインタ、配列、`sizeof`、`char`、文字列リテラル、グローバル変数、初期化式が入ると、単なる「整数を計算する言語」ではなく、実際の C コードらしいものを少しずつ受け付けられるようになります。

その一方で、型を意識しない設計ではもう限界が近いことも明らかになっているはずです。見た目が同じ `+` でも意味が違い、同じ宣言っぽい構文でも配置場所や初期化規則が変わるからです。次章では、まさにその限界を超えるために、型と制御構文をさらに本格的に育てていきます。

ステップ 29 以降

編集注記
この章以降は、公開ページ上で未完になっていた ステップ 29 以降 を補うために新規に書き下ろした続編です。既存の流れを保ちつつ、後半の実装方針を複数ステップへ分解して説明します。

ここまでの実装で、我々のコンパイラは整数、ポインタ、配列、関数、グローバル変数、文字列リテラル、初期化式をかなり扱えるようになっています。しかし、まだ C らしいプログラムを書くには足りないものが残っています。`typedef`、`struct`、`enum`、`switch`、`break`、`continue`、キャスト、複合代入、前置・後置インクリメントなどです。

これらは単に文法を足せばよいわけではありません。式や宣言がどの型を持っているかを一貫して扱えるようにしなければ、コード生成も意味解析もすぐ破綻します。したがって、後半の主題は「文法を増やす」こと以上に、「内部表現を育てる」ことになります。

ここから先の難しさ

前半までは、構文解析が終われば比較的素直にアセンブリへ落とせる場面が多くありました。ところが `p+1`、`x.y`、`(T)expr`、`a += b` のような構文が入ると、見た目だけでは処理を決められません。型、左辺値性、スコープ、宣言子、ラベルなど、多くの補助情報が必要になります。

つまり後半では、「入力文字列を読む」能力よりも、「読んだ結果をどう整理して保持するか」が重要になります。以後の各ステップは、表面構文を増やす話に見えながら、実際には内部表現を整備する話でもあります。

ステップ 29: 構文木に型を載せる

最初に着手すべきなのは、新しい構文ではなく型情報の導入です。各式ノードに `ty` のようなフィールドを持たせ、`int`、`char`、ポインタ、配列といった型を記録できるようにします。

なぜ最初に型なのか

後半の機能の多くは、式の型がわからないと正しく実装できません。sizeof はもちろん、ポインタ加算、代入の整合性確認、構造体メンバアクセス、キャスト、関数呼び出しの引数変換など、どれも型情報を前提にしています。したがって、型は後からその場しのぎで付け足すより、早めに構文木の一部へしてしまったほうがよいです。

どこで型を付けるか

実装としては、パーサが構文木を作ると同時に型を付けてもよいですし、パース後に別フェーズで木を歩いて型を計算しても構いません。教材としては前者のほうが実装量は少ないですが、後者のほうが責務が分かれて整理しやすいこともあります。どちらを選ぶにせよ、「各ノードが自分の型を持つ」状態を目標にするのが重要です。

必要になる型データ

最初に必要なのは、型の種類、サイズ、必要なら基底型です。配列なら要素型と要素数、ポインタなら基底型、関数なら戻り値型と引数型列、構造体ならメンバ一覧を持つことになります。後から struct や typedef が入ることを考えると、再帰的なデータ構造として型を表せるようにしておくべきです。

型エラー検出は後でもよい

この段階では、型エラーを厳密に全部検出する必要はありません。まずは「正しい入力に対して必要な型情報が得られる」ことを目標にしてください。誤り検出は後から少しずつ強化できます。最初から完璧な型チェッカを目指すと、後半の進行が大きく鈍ります。

ステップ 30: typedef と宣言子

次に欲しくなるのが typedef です。複雑な型へ名前を付けられるようになると、テストコードも実装も一気に読みやすくなります。特にポインタや構造体が増えると、別名なしでは宣言がすぐ読みにくくなります。

```
typedef int *IntPtr;
IntPtr p;
```

何が難しいのか

typedef を実装すると、識別子が「変数名」なのか「型名」なのかをスコープごとに区別しなければなりません。C ではこの区別は字句解析だけでは決まりません。過去に typedef された名前かどうかを、現在のスコープを見て判断する必要があります。

つまり、トークン列の意味が状態依存になります。これは前半の「文字列だけを見れば種類が決まる」世界から一歩進んだもので、C の複雑さが初めて強く表れる箇所の 1 つです。

宣言子が複雑になる理由

同時に、C の宣言子をきちんと扱う必要も出てきます。int *x[3]; のような宣言は左からでは読めません。基底型を用意し、宣言子部分を再帰的に読み進めて最終的な型を組み立てる専用の処理が必要です。

ここで宣言子専用の関数を用意しておくと、後の struct メンバ宣言や関数ポインタでもそのまま使い回せます。C の宣言は厄介ですが、厄介さが局所化されていると考えると多少気が楽になります。

typedef 名のスコープ

typedef 名もローカル変数と同様にスコープを持ちます。ブロックへ入ったら新しいスコープを作り、抜けたら破棄する必要があります。この仕組みが後のキャスト判定でも効きます。C では「今この識別子が何を意味するか」をスコープ抜きで語れない場面が多いのです。

ステップ 31: struct とメンバアクセス

構造体が入ると、コンパイラ自身の内部表現を C で素直に書けるようになります。トークン、型、構文木ノードのような複合データを、個別の配列ではなく 1 つのまとまりとして扱えるからです。セルフホストへ近づくためにも、構造体は非常に重要です。

構造体型が必要な理由

前半の世界では、1 個の値か 1 本のポインタだけを持つ世界でも何とか進めました。しかし実際のコンパイラ内部では、「種類」「文字列」「長さ」「行番号」を持つトークンや、「種類」「サイズ」「基底型」「メンバ」を持つ型情報など、複合データが大量に現れます。構造体は、そのような概念を自然に表現するための道具です。

実装上の中心はメンバ表

実装上は、メンバの名前、型、オフセット、必要ならアラインメントを保持する構造体型を導入します。p.x ではオフセット付きメモリアクセスを生成し、p->x は (*p).x として扱えば整理しやすくなります。

```
struct Point {
  int x;
  int y;
};
```

このとき必要なのは、「x は先頭から何バイト目にあるか」という情報です。したがって、メンバ表は名前の一覧であるだけでなく、レイアウト表でもあります。

パディングとアラインメント

ここで重要なのはレイアウト計算です。char のあとに int を置くとき、通常はパディングが入ります。sizeof やメンバアクセスを正しくするには、ABI に従ったサイズと整列を計算しなければなりません。

たとえば次のような構造体を考えます。

```
struct T {
  char a;
  int b;
};
```

この場合、b を 4 バイト境界へ揃えるために a のあとへ空きが入るのが普通です。こうした「見えない隙間」が存在することを、コンパイラは正しく理解していなければなりません。

. と ->

. は構造体値のメンバ参照、-> はポインタが指す構造体のメンバ参照です。後者は内部的には (*p).x と見なせます。この還元を使えば、実装を共有しやすくなります。C の複雑な構文の多くは、内部表現で核となる少数の概念へ落とし込めます。

ステップ 32: enum、switch、case

enum は構造体より単純です。列挙子は整数定数として扱えるので、名前と値の対応を記録すれば十分です。これが入るとトークン種別やノード種別を C 側のコードでも自然に表現できます。

enum の意義

実装としては簡単でも、enum の導入には大きな価値があります。整数定数を裸で使うのではなく、意味のある名前を持たせられるからです。コンパイラ自身のコードを整理するうえでも、この恩恵は大きいです。

switch が欲しくなる理由

続いて switch 文を入れると、多分岐がかなり書きやすくなります。トークン種別やノード種別に応じて処理を分けるコードは、コンパイラ内部でも大量に出てきます。if の連鎖より switch のほうが自然な場面は多く、セルフホストを見据えると早めに欲しくなる機能です。

実装の骨格

実装は「条件式を一度評価し、各 case の定数と比較して一致するラベルへ飛ぶ」という形で整理できます。default の扱いと、break の飛び先管理が追加の論点です。

```
switch (x) {
case 0:
  return 1;
case 1:
  return 2;
default:
  return 3;
}
```

単純な実装では、各 case を順に比較して一致したラベルへ飛べば十分です。ジャンプテーブルのような最適化は後回しで構いません。

ステップ 33: break、continue、goto

break と continue はループや switch を実用的にするうえで不可欠です。現在有効な break 先ラベルと continue 先ラベルを管理すれば、コード生成は比較的素直に書けます。

文脈付きジャンプ

ここで難しいのは、break と continue の飛び先が「今どの構文の中にいるか」で変わることです。switch 中の break は switch の末尾へ飛び、while 中の continue はループ条件や増分位置へ飛びます。したがって、パーサかコード生成器が現在有効なジャンプ先を管理しておく必要があります。

goto の前方参照

goto はさらに単純で、ラベル名からアセンブリラベルへの対応を持てば実装できます。ただし前方参照があるので、関数読み込み後に未定義ラベルがないか確認する仕組みがあると安全です。

```
goto end;
...
end:
  return 0;
```

ラベル定義より前に goto が現れる可能性がある以上、その場で完全に解決することはできません。この「後で照合する」設計は、リンカの未解決シンボル処理と少し似ています。

なぜ goto を入れるのか

goto はしばしば嫌われますが、コンパイラ自身のコードを書く立場になると、エラー処理や状態機械の実装で役立つ場面があります。C らしいコードを無理なくコンパイルできるようにするためにも、早めに入れておく価値があります。

ステップ 34: キャストと void

キャストが入ると、低レイヤ寄りの C コードがかなり通るようになります。特に void * や整数型同士の変換は、ライブラリやシステムプログラミングで頻出です。

パース上の難しさ

難しいのは、(foo)bar がキャストなのか、単なる括弧付き式なのかを見分ける点です。ここでも typedef 名を識別できる仕組みが効きます。foo が型名として見えるならキャスト、そうでなければ括弧式として扱う、という判断が必要になります。

この問題は C の典型的な「字句だけでは決まらない構文」です。typedef を先に入れておく理由がここで効いてきます。

void の最小実装

void はまず「値を持たない型」として導入し、厳密な型規則は後から詰めれば十分です。void * は任意のオブジェクトポインタとやりとりできる型として扱っておくと、実用的なコードがかなり通るようになります。

変換命令の実際

整数の幅が変わるキャストでは、ロードや拡張の方法が変わります。ポインタと整数のあいだを行き来するキャストも、少なくともサイズ上の整合は意識しなければなりません。このあたりは厳密にやり始めると細かい規則が多いですが、最初は代表例が通ることを重視して構いません。

ステップ 35: 複合代入と前置・後置インクリメント

+=、-=、++、-- は見た目以上に罠が多い構文です。左辺を一度しか評価してはいけないため、単純なマクロ展開のように書き換えると副作用を重複評価してしまう危険があります。

何が危険なのか

たとえば *f() += 1 を単純に *f() = *f() + 1 へ展開すると、f() を 2 回呼んでしまいます。C では左辺式の副作用が重要なので、これは許されません。したがって、左辺値のアドレスを 1 回だけ計算し、そのアドレスへ対してロード・演算・ストアを行う必要があります。

実装の中心

そのため、左辺値のアドレスを一度だけ計算し、そこへロード・演算・ストアを行う共通処理を持つのがよい設計です。ポインタに対する p++ や p += 1 では、要素サイズを掛けるという従来の規則も忘れず適用します。

前置と後置の違い

後置インクリメント x++ は、「式の値は更新前、変数の中身は更新後」でなければなりません。前置インクリメント ++x は、「式の値も更新後」です。見た目は近くても、式の値として何を返すかが違うため、内部ノードの種別を分けておいたほうが実装しやすくなります。

ステップ 36: 関数宣言と型チェックの強化

ここまで来ると、かなり多くの C コードが通るはずですが、セルフホストや外部コードを意識するなら関数プロトタイプ、暗黙変換、整数拡張、代入や比較の整合性チェックも少しずつ強化したくなります。

ここからの主題

この段階では、パースと意味解析を分ける意義が大きくなります。構文解析は木を作ることに集中し、型付けや整合性確認は別フェーズで行うようにすると、その後の `union`、可変長引数、浮動小数点なども拡張しやすくなります。

関数プロトタイプ

関数定義だけではなく宣言も扱えるようになると、引数型や戻り値型を事前に知ったうえで呼び出しをチェックできます。これにより、セルフホスト用の大きなコードでも意味不明な崩れ方をしにくくなります。型情報は、後半では正しさと開発効率の両方を支えます。

暗黙変換と整合性

C は暗黙変換の多い言語です。小さい整数型は `int` へ持ち上がり、配列は多くの場面でポインタへ崩れ、代入や比較では共通の型へ寄せられます。こうした規則を少しずつ導入し始めると、コンパイラは「ただ構文が通る」だけでなく、「C らしく意味を理解する」段階へ近づきます。

ここまで実装できると何が嬉しいのか

`struct`、`typedef`、`switch`、複合代入、キャストが入ると、コンパイラ自身のコードを C で自然に書ける範囲が大きく広がります。トークナイザ、型システム、パーサの内部データ構造を素直に表現できるようになるからです。

後半の本質は、文法を増やすこと自体ではなく、コンパイラの内部データ構造を C の複雑さに耐えられるよう育てることです。そこまで到達すれば、セルフホストや実用機能への道筋がかなり見えてきます。

その先に残るもの

もちろん、ここまで実装してもまだ完全な C には届きません。プリプロセッサ、`union`、浮動小数点数、可変長配列、可変長引数関数、ビットフィールド、ABI に従った複雑な値渡しなど、本気で実用処理系を目指すならさらに多くの要素が残っています。

しかし、それらへ進むための土台は、この章まででかなり揃います。ここから先は「新しい表面構文を増やす」以上に、「既存の内部表現をどこまで拡張に耐えられるように育てるか」が主題になります。

後半機能のテスト戦略

後半は機能ごとの依存が強くなるため、テストも段階的に増やす必要があります。おすすめは、各ステップごとに「その機能だけを確認する最小コード」と、「既存機能と組み合わせた少し長いコード」の両方を用意することです。

たとえば `struct` を入れたら、まずはメンバ代入と参照だけの最小例を作り、その次に配列やポインタと組み合わせた例を足します。`switch` を入れたら、`break` を含まない最小例と、`break` と `default` を含む例を分けて置きます。複合代入や `++` では、副作用が 1 回だけ評価されるかを確認する専用テストが重要です。

後半のバグは、入力が少し長くなると急に出るものが多いので、短いテストと少し長いテストの両輪が効きます。型、スコープ、ラベル、左辺値が絡むため、1 個の修正が複数機能へ波及しやすいからです。

スタティックリンクとダイナミックリンク

リンカには、必要なコードを実行ファイルへ直接取り込む静的リンクと、実行時に共有ライブラリを読み込む動的リンクがあります。どちらもシンボル解決という意味では同じ問題を扱っていますが、配置される場所とタイミングが異なります。

静的リンク

静的リンクでは、必要なライブラリコードを最終実行ファイルへ取り込みます。完成した実行ファイル 1 個で配布しやすく、実行時依存も減ります。その代わり、バイナリは大きくなりやすく、ライブラリ更新を取り込むには再リンクが必要です。

教材実装としては、静的リンクのほうが仕組みを理解しやすいです。必要なシンボルが最終的にすべて 1 個の実行ファイルへ集まるため、何がどこから来たのかを追いやすいからです。

動的リンク

動的リンクでは、共有ライブラリを実行時に読み込みます。複数プロセスで同じライブラリコードを共有でき、ライブラリ更新も柔軟になります。一方で、実行時ローダ、位置独立コード、PLT/GOT のような仕組みが必要になり、見通しは少し複雑になります。

コンパイラ学習の段階では、まず静的なシンボル解決を理解し、そのあとで「外部参照を実行時まで遅らせる仕組み」があると捉えるのがわかりやすいです。いきなり動的リンクの細部へ入ると、本筋からかなり外れます。

コンパイラとの関係

コンパイラ自身は、静的リンクと動的リンクのどちらで最終的に結ばれるかを、常に深く意識している必要はありません。必要なのは、外部シンボル名を正しく出し、呼び出し規約を守ることです。ただし、どこまでがコンパイラの責務で、どこからがリンカやローダの責務なのかを理解しておくことは非常に重要です。

C の型の構文

C の型構文は、宣言と利用の見た目が似ていることで知られています。`int *x[3]` は「`x` は 3 要素の配列で、その各要素は `int *`」という意味です。読むコツは、識別子から外へ向かって優先順位の高いものからたどることです。

なぜ読みづらいのか

多くの言語では型が比較的単純な並びになりますが、C ではポインタ、配列、関数が宣言子の周囲へまとわりつくような見た目になります。人間にも読みにくいということは、当然コンパイラにも専用の処理が必要だということです。

宣言子を再帰的に読む

配列、ポインタ、関数、`struct`、`typedef` が増えると、型を単なる文字列として処理するのは無理になります。基底型から出発し、宣言子を再帰的に適用して最終型を構築するという内部表現が必要です。

```
int *x[3];
```

この宣言では、まず x が識別子であり、その外側に配列 [3] が付き、さらにその要素型が int * です。左から読むと見失いやすいので、識別子を中心に考える習慣を持つと理解しやすくなります。

実装上の利点

この章は一見文法の話ですが、実際にはパーサ設計そのものの話です。C の宣言は複雑でも、内部表現を素直に作っておけば実装は意外と整理できます。逆に、宣言子の処理を都度 ad hoc に書き始めると、typedef、関数引数、構造体メンバで同じような複雑さを何度も相手にする羽目になります。

読み方の練習

宣言を読む練習としては、短い例を紙に書いて「識別子から外へ読む」ことを繰り返すのが効果的です。

```
int *x[3];
int (*fp)(int);
char (*table[4])[8];
```

こうした宣言を人間が読めるようになると、コンパイラがなぜ宣言子専用の再帰関数を必要とするのかも自然に理解できます。C の型構文は慣れの要素が強いので、短い練習をいくつか入れるだけで印象がかなり変わります。

おわりに

小さな式コンパイラから始めて、制御構文、関数、ポインタ、配列、グローバル変数、型システムへと進む流れを追うと、コンパイラは決して神秘的な道具ではないことがわかります。大きなシステムも、小さな整合した部品の積み重ねです。

重要なのは、常に動くものを保ちながら少しずつ前進することです。コンパイラ作成は難しい分野に見えますが、その難しさの多くは「一度に全部見ようとすること」から生まれます。1 ステップずつ分解すれば、むしろ非常に気持ちのよい分野です。

もうひとつ大切なのは、途中で設計を育て直すことを恐れないことです。前半で十分だった実装が、後半では窮屈になることは普通にあります。型情報を構文木へ載せる、スコープを導入する、左辺値処理を共通化する、といった変更は後退ではなく成長です。コンパイラ実装は、言語仕様と内部表現の対話でもあります。

付録 1: x86-64 命令セット チートシート

この付録は、前半でよく出てくる命令の最小限の見取り図です。命令名を暗記することが目的ではなく、「何をしたいときにどの種類の命令が現れるか」を把握するためのメモだと考えてください。

整数演算

- mov: 値を移動する。レジスタ間、メモリとレジスタのあいだで頻出する。
- add, sub: 加算と減算。もっとも基本的な算術命令。
- imul, idiv: 乗算と除算。除算は使い方に少し癖がある。
- neg: 符号反転。単項マイナスの実装で見かけることがある。
- cmp: 比較してフラグを立てる。条件分岐や比較演算の前段として使う。

メモリアクセス

- push, pop: スタック操作。式評価の一時退避やプロローグで使う。

- `lea`: 実メモリアクセスを伴わずアドレス計算する。ポインタやローカル変数アドレス計算で便利。
- `mov [addr], reg`: メモリへのストア。
- `mov reg, [addr]`: メモリからのロード。

分岐と条件コード

- `jmp`: 無条件ジャンプ。
- `je`, `jne`, `jl`, `jle`: 条件ジャンプ。直前の比較結果のフラグを見る。
- `sete`, `setne`, `setl`, `setle`: 条件成立時に 1 を書き込む。比較結果を 0/1 の値へ落とすのに便利。

関数呼び出し

- `call`: 呼び出し先へ飛び、復帰アドレスを積む。
- `ret`: 呼び出し元へ戻る。
- `push rbp; mov rbp, rsp`: 典型的なプロローグ。
- `mov rsp, rbp; pop rbp`: 典型的なエピローグ。

覚え方のコツ

生成されたアセンブリを読むときは、1 命令ずつ完全に理解しようとするより、「今どの値をどこへ動かしているか」「次にどこへ飛びたいのか」という観点で追うと理解しやすくなります。命令セットは丸暗記する対象ではなく、動いている値の流れを観察するための語彙です。

付録 2: Git によるバージョン管理

コンパイラのような段階的開発では、バージョン管理が非常に重要です。各ステップごとにコミットを切っておけば、「どこで壊れたか」をすぐに戻って確認できます。教材としても、自分の理解の履歴がそのまま残るので価値があります。

おすすめのコミット単位

おすすめは、各ステップを次の単位で分けることです。

- 仕様変更
- 実装
- テスト
- 必要なら軽いリファクタ

この単位に分けておくと、後で履歴を読むときに「このコミットは何を変えたのか」がすぐにわかります。コンパイラ実装では、仕様の追加、内部表現の変更、テストの増補が密接に絡むため、コミット粒度の良し悪しが学習効率へ直結します。

履歴が仕様書になる

履歴を細かく保つと、後で記事や発表資料にまとめるときにも役立ちます。ある機能を入れるのに何を先に変え、何がボトルネックだったかが、そのまま説明資料の骨格になるからです。コンパイラ作成は段階の積み重ねなので、Git の履歴と非常に相性がよい分野です。

付録 3: Docker を使った開発環境の作成

ツールチェインの差異で悩みたくないなら、Docker で開発環境を固定すると便利です。C コンパイラ、`make`、`binutils`、テストスクリプトをまとめたイメージを 1 つ用意しておけば、どのマシンでも同じ手順で検証できます。

なぜ環境固定が効くのか

コンパイラ作成では、アセンブラやリンカ、標準ライブラリ、`objdump` の挙動が環境によって少しずつ違うことがあります。教材として配布する場合、環境構築でつまずかないことは内容理解と同じくらい重要です。環境差を吸収できるだけで、学習コストがかなり下がります。

最小構成の考え方

最初から大規模な開発環境を作る必要はありません。C コンパイラ、`make`、`binutils`、必要なら `bash` と簡単なテストスクリプトがあれば十分です。コンパイラ本体だけでなく、再現可能な開発環境をセットで考える習慣を持つと、長い開発でも疲れにくくなります。

付録 4: テストケースの育て方

コンパイラのテストは、後からまとめて大量に書くより、各ステップで少しずつ育てていくほうが圧倒的に楽です。新しい機能を入れるたびに、その機能だけを確認する最小例と、既存機能と組み合わせた例を追加していくのが基本です。

たとえば `if` を入れた直後なら、真分岐だけ、偽分岐だけ、`else` あり、ネストあり、という 4 種類程度をまず揃えます。ポインタを入れたら、アドレス取得、デリファレンス、代入、ポインタ加算をそれぞれ独立に確認します。構造体を入れたら、メンバ参照、ポインタ経由参照、`sizeof`、パディングが効く例を用意するとよいです。

テストコードは小さいほど原因を切り分けやすくなります。大きなサンプルは後から追加すれば十分で、最初は 1 機能 1 論点の短いコードを優先したほうがよいです。コンパイラ開発では、短いテストの積み重ねがもっとも強い安全網になります。

付録 5: ステップ別サンプルプログラム

この付録では、本編の各段階で試してみると理解が深まる短いサンプルを並べます。すべてをそのまま使う必要はありませんが、「この時点でどこまで通ればよいのか」を掴む目安として役立ちます。

式だけの段階

最初の数ステップでは、まず優先順位と括弧が正しく効くかを確認します。たとえば次の入力は、四則演算と構文木が崩れていないかを見る基本セットです。

```
1 + 2 * 3
(1 + 2) * 3
10 - 3 - 2
-5 + 8
```

ここで確認したいのは、結果の数値だけではありません。生成されたアセンブリを見て、左辺を積み、右辺を計算し、最後に演算しているかも一緒に観察すると、コード生成の感覚が定着します。

比較演算を入れたあとは、次のような入力を加えると層構造が正しく機能しているかを確認しやすくなります。

```
1 == 1
1 != 2
1 < 2
2 <= 2
1 + 2 < 4
```

比較演算は単純に見えて、トークナイザの最長一致、文法の優先順位、コード生成の条件設定が一度に関わります。短い入力でも多くの要素を確認できる、効率のよいテストです。

文、変数、関数の段階

文を導入したら、まずは `return` とローカル変数を含む小さな関数から確認するとよいです。

```
int main() {
  int a;
  a = 3;
  return a;
}
```

この例では、ローカル変数の領域確保、代入、ロード、戻り値設定までを一度に確認できます。値が通ることに加え、スタックフレーム上のオフセットがずれていないかを見るのにも向いています。

制御構文を確認するなら、次のような短い例が使いやすいです。

```
int main() {
  int x;
  x = 0;
  if (1)
    x = 3;
  return x;
}
```

```
int main() {
  int x;
  x = 0;
  while (x < 5)
    x = x + 1;
  return x;
}
```

`if` と `while` は、どちらもラベルとジャンプの配置を確認するのに向いています。生成されたコードが読みやすい段階のうちに、どのラベルが何を意味しているかを自分で追ってみるとよいです。

関数呼び出しを入れたら、自分で定義した関数と外部関数の両方を試してみると、呼び出し規約の理解が深まります。

```
int inc(int x) {
  return x + 1;
}

int main() {
  return inc(41);
}
```

ポインタ、配列、グローバル変数の段階

ポインタを入れたあとは、アドレス取得とデリファレンスが正しく往復しているかを最初に確認します。

```
int main() {
  int x;
  int *p;
  x = 3;
  p = &x;
  return *p;
}
```

この例は短いですが、左辺値、アドレス計算、ロードのすべてが入っています。ここが安定すると、以後の `*p = 3` やポインタ加算も追いやすくなります。

配列については、添字演算子が `*(base + index)` へ落ちているかを意識して観察すると理解が深まります。

```
int main() {
  int x[3];
  x[0] = 4;
  x[1] = 5;
  x[2] = 6;
  return x[1];
}
```

文字列リテラルを導入したら、静的領域へデータが置かれていることを確認するために、短い文字列と `char` の取り出しを試すとよいです。

```
int main() {
  char *s;
  s = "abc";
  return s[1];
}
```

グローバル変数と初期化式は、ローカル変数と意図的に対比すると違いが見えやすくなります。

```
int g = 3;

int main() {
  int x = 4;
  return g + x;
}
```

この例では、`g` がデータセクション側、`x` がスタック側に住むことを意識すると、配置の違いがはっきり見えます。

型と後半機能の段階

後半に入ったら、`typedef` と `struct` は早めに最小例を通しておくと設計が安定します。

```
typedef int *IntPtr;

int main() {
  int x;
  IntPtr p;
  p = &x;
  *p = 7;
```

```
  return x;
}
struct Point {
  int x;
  int y;
};

int main() {
  struct Point p;
  p.x = 3;
  p.y = 4;
  return p.x + p.y;
}
```

switch、break、continue の確認には、値が少し変化する短い反復コードが向いています。

```
int main() {
  int x;
  int y;
  x = 0;
  y = 0;
  while (x < 5) {
    x = x + 1;
    if (x == 2)
      continue;
    y = y + x;
  }
  return y;
}
```

複合代入や後置インクリメントは、副作用の回数を意識して確認する必要があります。

```
int main() {
  int x;
  x = 3;
  x += 4;
  x++;
  return x;
}
```

このあたりのテストは短くても、実装上は左辺値処理や型情報が関わります。見た目より多くのものを一度に検証できるので、回帰テストとしても優秀です。

付録 6: 実装チェックリスト

最後に、各段階で見落としやすい点を簡単なチェックリストとしてまとめておきます。新しい機能を入れたあとにざっと見返すだけでも、かなりの事故を防げます。

字句解析

- 2 文字演算子を 1 文字演算子より先に読んでいるか
- 空白とコメントを確実に読み飛ばしているか
- エラー位置を元の入力へ対応付けられるか
- 識別子と予約語を区別できるか

構文解析

- 文法の優先順位がコードの関数分割へ反映されているか

- 左再帰を避けた形になっているか
- 宣言と式の分岐を文脈に応じて選べるか
- `typedef` 名と変数名をスコープ込みで判定できるか

構文木と型

- 各式ノードが自分の型を持っているか
- 左辺値かどうかを必要な箇所で区別できるか
- ポインタや配列の基底型情報を保持しているか
- 構造体メンバに名前とオフセットが付いているか

コード生成

- 関数プロローグとエピローグでスタックが対になっているか
- 条件分岐のラベル生成が衝突していないか
- `break`、`continue`、`goto` の飛び先を文脈ごとに管理しているか
- 複合代入や `++` で左辺を複数回評価していないか

ビルドと検証

- 小さなテストと少し長い統合テストの両方があるか
- 生成したアセンブリを数本は人間の目で追っているか
- 外部関数呼び出し時のスタック整列を確認しているか
- 失敗時のエラーメッセージが次の修正へつながる情報を持っているか

付録 7: デバッグの進め方

コンパイラのデバッグは、生成結果が多段になっているぶん、通常のアプリケーションより切り分けの視点が多くなります。逆に言えば、段階ごとに観察点を分ければかなり整理して追えます。この付録では、どの層をどう疑うと進めやすいかをまとめます。

字句解析で疑うべきこと

最初に確認すべきは、トークン列が期待通りにできているかです。パーサがおかしいように見える不具合でも、実際にはトークナイザが `<=` を `<` と `=` に分けていたり、コメントを飛ばし損ねていたりすることがよくあります。

このため、トークナイザには早い段階でダンプ関数を付けておくと便利です。失敗した入力をまずトークン列として眺めるだけで、原因が字句レベルか構文レベルかをかなり早く切り分けられます。識別子や数値に元の位置情報を持たせておけば、エラー報告もそのまま改善できます。

構文解析で疑うべきこと

パーサの不具合では、文法の層と現在読んでいるトークン位置を目で追えるようにしておくのが有効です。`expr()` に入った、`mul()` に降りた、`primary()` で整数を読んだ、といったログが少しあるだけで、どこで期待と実際がずれたかが見えます。

とくに C のような言語では、「ここは宣言なのか式なのか」「これは型名なのか識別子なのか」という分岐が増えます。したがって、分岐直前の状態を出力できるだけでデバッグ効率がかなり変わります。

型と意味解析で疑うべきこと

後半の不具合では、構文木自体は正しくできているのに、型付けや左辺値判定がずれていることがよくあります。`p+1` が整数加算として処理されていたり、`*p` が左辺値になっていなかったりすると、コード生成で奇妙な振る舞いになります。

こうした問題に備えて、構文木ダンプへ型情報も一緒に出すと有効です。各ノードが何型か、左辺値として扱われるか、どの識別子へ解決されたか、といった情報を見える化しておくと、後半のデバッグはかなり楽になります。

コード生成で疑うべきこと

生成されたコードが壊れているときは、「式の意味が違う」のか「スタックやレジスタ管理が崩れている」のかをまず分けて考えるとよいです。前者なら型や構文木、後者ならプロローグ、エピローグ、一時値退避、呼び出し規約を疑うべきです。

スタックを使う実装では、`push` と `pop` が最終的に釣り合っているかを見るだけでもかなりの不具合を発見できます。関数呼び出し直前のスタック整列も、外部関数が絡む不具合では必ず確認したい点です。

失敗する入力を小さくする

不具合に遭遇したとき、最初の入力が長いまま追いかけるのは得策ではありません。再現に必要な最小入力まで削るだけで、原因の見通しが一気に良くなります。コンパイラでは、入力を 1 行ずつ削るだけで、責任範囲の大半を切り落とせることがよくあります。

この「失敗例の最小化」は、通常のアプリケーション以上に強力です。入力言語があるということは、失敗を再現する文字列を人間が直接編集できるということだからです。

付録 8: セルフホストへ向かう順序

コンパイラ作成の大きな節目として、セルフホストがあります。つまり、自分のコンパイラ自身を自分のコンパイラがコンパイルできる状態です。ここへ到達するには、単に文法を増やすだけでなく、コンパイラ自身のコードが自然に書けるだけの表現力を揃える必要があります。

まず必要なもの

セルフホストへ向かう前に最低限欲しいのは、関数、ローカル変数、ポインタ、配列、文字列リテラル、グローバル変数、`struct`、`typedef`、`enum`、`switch` あたりです。コンパイラ内部ではトークンや型のような複合データを大量に扱うため、構造体なしではかなり不自然になります。

また、エラーメッセージや文字列処理のために `char *` と文字列リテラルが安定していることも重要です。セルフホストは機能一覧を埋める作業というより、コンパイラ自身のコードが「無理なく書けるか」を基準に考えるほうがうまくいきます。

ブートストラップの考え方

一般には、最初に既存の C コンパイラで自作コンパイラをコンパイルし、その生成物でもう一度自分自身をコンパイルし直す、という手順を取ります。この 2 回目と 3 回目の出力が安定してくると、セルフホストへかなり近づいたと判断できます。

ここで重要なのは、いきなり巨大なソースを通そうとしないことです。まずはトークナイザだけ、次にパーサの一部だけ、最後に全体というように、コンパイラ自身を構成する小さな部品ごとに移植していくほうが失敗しにくくなります。

セルフホストで露呈しやすい弱点

セルフホストへ進むと、小さなサンプルでは見えなかった弱点が一気に露呈します。型の曖昧さ、スコープ処理の漏れ、複合代入の副作用、ラベル管理、エラー処理の弱さなどです。これらはどれも、短いサンプルでは偶然動いていた実装が、大きなコードで破綻する典型的な箇所です。

その意味で、セルフホストはゴールであると同時に、実装の粗さを洗い出す巨大なテストでもあります。到達できれば終わりではなく、そこから初めて「自分の実装のどこが本当に弱いか」が見えます。

無理なく近づくために

セルフホストを急ぎすぎると、途中で入れた雑な実装が大量の負債になります。前半で小さく動かし、後半で内部表現を育て、最後に大きな入力へ挑む、という順序を守ったほうが結果的には速いです。コンパイラ作成では、遠回りに見える整備こそが最短距離になることがよくあります。

付録 9: よくあるつまずきと考え方

この付録では、コンパイラを書いていると頻繁に遭遇する「何かがおかしいが、どこを見るべきかわからない」状況をいくつか整理します。どれも一度は通る場所であり、前もって知っているだけでかなり気が楽になります。

パーサが壊れて見えるが、実はトークナイザの問題

もっともよくあるのはこれです。`<=` が `<` と `=` に分割されていたり、コメント末尾の処理を誤っていたりすると、見た目にはパーサが変な場所で失敗したように見えます。しかし原因はその前段にあります。

この種の不具合では、まずトークン列を出力してから考える癖を付けるのが重要です。構文木やコード生成を疑うのは、その後で十分です。コンパイラは多段の変換器なので、前段の崩れが後段で奇妙な形に見えることは日常的に起こります。

生成コードが壊れて見えるが、実は左辺値処理の問題

`*p = 3`、`a[i] = x`、`x.y = z` といった式が絡むと、「コード生成が間違っている」と感じる場面が増えます。しかし実際には、左辺値として扱うべき式を早い段階で値へ潰してしまっていることが原因である場合が多いです。

値と場所を区別する設計が曖昧だと、代入、アドレス取得、デリファレンス、構造体メンバアクセスのすべてが不安定になります。後半へ進むほど、この区別の重要性は強まります。

構文は通るが意味が変になる

後半では「パースは成功するが結果が変」という不具合が増えます。これは型付けや暗黙変換、スコープ解決の問題であることが多いです。たとえば `p+1` を整数加算として扱っていたり、`typedef` 名を変数名として解釈していたりすると、このタイプの崩れ方をします。

この段階では、構文木ダンプへ型や解決済みシンボル情報を出せるようにしておくのが有効です。見えない情報ほど、出力できるようにした瞬間に不具合が単純化します。

ある入力では動くが、少し長い入力で急に壊れる

これは局所的な実装が偶然うまく噛み合っていた典型です。スタック整列、ラベルの一意性、スコープ破棄の漏れ、未初期化データなどが原因になりやすく、短い入力では隠れてしまいます。

したがって、各機能について「最小例」と「少し長い組み合わせ例」の両方を持っておくのが重要です。短い入力だけで安心すると、後で大きな入力を通した瞬間にまとめて問題が噴き出します。

エラー報告が弱いと進まなくなる

実装初期には、エラー報告はつい後回しになります。しかし、後半に入ると診断の弱さがそのまま開発速度の低下になります。型名か式か、ラベルが未定義か、構造体メンバが存在しないか、といった区別を出せるかどうかで、1 つの不具合にかかる時間が大きく変わります。

エラーメッセージはユーザー向け機能であると同時に、自分自身のためのデバッグ装置でもあります。教材実装であっても、少しずつ整える価値があります。

設計を途中で作り直すのは普通

前半で十分だった設計が、後半で窮屈になるのは自然です。最初から型情報を完璧に持たせたり、宣言子を完全対応したりする必要はありません。必要になった段階で内部表現を育て直すことは、失敗ではなく正常な進行です。

コンパイラ作成は、1 回で正解の設計を当てるゲームではありません。何が足りないかを機能追加のたびに学び、そこで内部表現を少しずつ鍛えていく作業です。

付録 10: 章ごとの復習ポイント

最後に、本書全体を通して何を掴めればよいかを章ごとに短く振り返ります。読み終えたあとや、数日空けて戻ってきたときの目印として使ってください。

導入と理論

最初の章で押さえるべきなのは、「コンパイラは入力を別の形式へ変換するプログラムであり、字句解析、構文解析、構文木、コード生成という流れで捉えると理解しやすい」という感覚です。プッシュダウンオートマトンや再帰下降構文解析は、その流れを支える理論的背景として位置付ければ十分です。

式だけのコンパイラ

前半の実装章では、四則演算と比較演算を通じて、文法の層がそのままパーサ関数の層になること、構文木がコード生成を単純にすること、アセンブリ出力はデバッグしやすいことを体験できれば成功です。

文、変数、関数

中盤では、式だけの世界から、状態と制御を持つプログラムの世界へ移ります。ローカル変数、左辺値、スタックフレーム、関数呼び出し、ABI といった概念がここで一気につながります。コンパイラが外部環境と約束事を共有していることも、この段階で見えるようになります。

ポインタと配列

ポインタ以降では、「型がコード生成へ直接効く」ことを体感できるはずです。`p+1` が整数加算ではないこと、配列とポインタが似ているが同じではないこと、文字列リテラルが静的データであることなど、C らしい癖の多くがここにあります。

後半の型と内部表現

最後の後半では、`typedef`、`struct`、`switch`、キャスト、複合代入などを通じて、コンパイラの内部表現を育てる必要性が前面に出ます。ここで掴むべきなのは、文法を増やすこと自体よりも、型、スコープ、ラベル、左辺値、レイアウト情報といった「見えないデータ」をどう整理するかです。

その先へ進むために

ここまで来たら、次に何を実装すべきかはかなり自分で判断できるようになっているはずです。プリプロセッサへ進むのか、`union` や浮動小数点数へ進むのか、セルフホストを目指すのか。どの方向へ進むにしても、本書で育てた「小さく作り、動かし、内部表現を必要に応じて強くする」という姿勢がそのまま役立ちます。

付録 11: 用語小事典

最後に、本書で頻出した用語を短い解説付きでまとめます。後半になるほど、表面構文より内部用語の意味が重要になるため、ここを言葉として持っておくと読み返しやすくなります。

AST

抽象構文木です。入力文字列の見た目ではなく、意味上の構造を木として表したものです。コンパイラの後半では、型付け、左辺値判定、コード生成などのほとんどが AST 上で行われます。

字句解析

ソースコードをトークン列へ分解する段階です。空白やコメントを飛ばし、数値、識別子、記号を種類付きの単位へまとめます。ここが安定すると、以後の処理の見通しが急に良くなります。

構文解析

トークン列を文法に従って読み、AST を作る段階です。再帰下降構文解析では、文法の非終端記号ごとに関数を書く形になります。前半ではもっとも「コンパイラを書いている感じ」が出る部分です。

意味解析

構文としては正しい入力へ、型やスコープなどの意味情報を付ける段階です。C では後半に入るほどこの役割が重要になります。単に木が作れたかだけでは、正しいコード生成へ進めないからです。

左辺値

書き込み可能な場所を表す式です。変数そのもの、デリファレンス、構造体メンバ参照などが典型例です。値と場所を区別する感覚は、C コンパイラを書くうえで最重要の 1 つです。

右辺値
評価結果として得られる値を表す式です。コンパイラの実装では、左辺値から必要に応じてロードして右辺値へ変換する、という流れがよく現れます。

スコープ
名前が有効な範囲です。ローカル変数、`typedef` 名、構造体タグなどはスコープを持ちます。後半では「同じ綴りでも今どの意味か」を判定するために欠かせません。

ABI
関数呼び出し規約やデータ配置など、外部世界との約束事です。コンパイラが外部関数や他言語のコードとやりとりできるのは、ABI を守っているからです。

再配置
最終アドレスが未定の参照を、リンク時に埋め直す仕組みです。オブジェクトファイルとリンカを理解するうえで中核になります。コンパイラの外側にある仕組みですが、外部シンボル呼び出しの理解には欠かせません。

セルフホスト
自分自身を自分のコンパイラでコンパイルできる状態です。大きな節目ですが、目的は名目上の到達だけではなく、自分の実装の弱点を一気に露呈させる大規模テストでもあります。

宣言子
C の宣言において、識別子の周囲に付く `*`、`[]`、`()` などの部分です。C の型構文が読みづらい理由の大半は宣言子にあります。実装では、宣言子専用の再帰的な処理を持つのが一般的です。

暗黙変換
言語が明示的なキャストなしに自動で行う型変換です。C では整数拡張、配列からポインタへの変換、比較や代入時の型合わせなどが代表例です。後半の意味解析では、この規則の導入が大きなテーマになります。

パディング
構造体メンバの整列を満たすために挿入される空き領域です。C プログラムからは見えにくいですが、`sizeof` やメンバオフセットを正しくするためにコンパイラは必ず理解していなければなりません。

スタックフレーム
関数呼び出しごとに使われるスタック上の領域です。ローカル変数や一時退避、保存レジスタなどがここに置かれます。ローカル変数実装の理解は、そのままスタックフレーム理解につながります。

トークン
字句解析の結果として得られる、意味のある最小単位です。数値、識別子、記号、文字列リテラルなどがトークンとして表現されます。トークン化が安定すると、パーサは文字列の細部を意識せずに済むようになります。

予約語

`if` や `return` のように、特別な意味を持つ識別子です。見た目は通常の単語と似ていますが、変数名としては使えません。C では `typedef` 名との区別も絡むため、識別子処理は想像以上に奥行きがあります。

オフセット

ある基準位置から何バイト離れているかを表す値です。ローカル変数なら `rbp` からの距離、構造体メンバなら構造体先頭からの距離として現れます。C コンパイラでは「名前」を最終的に「オフセット」へ落とし込む場面が非常に多いです。

テキストセクション

機械語のコードが入る領域です。関数本体など、実際に CPU が命令として実行するものがここに置かれます。データ領域と区別して考えるだけで、グローバル変数や文字列リテラルの配置が理解しやすくなります。

データセクション

初期値つきの静的データが入る領域です。グローバル変数や文字列リテラルの多くがここへ置かれます。実行前に内容が確定している、という点がローカル変数との大きな違いです。

BSS

ゼロ初期化される静的領域を表すセクションです。ファイル上へ大量のゼロを書き込む代わりに、「この大きさの領域をゼロで確保せよ」という情報として表されることが多いです。コンパイラ本体では見えにくいですが、グローバル変数の実行イメージを考えるときに役立ちます。

プロローグ

関数先頭でスタックフレームを準備する命令列です。古いフレームポインタの退避やローカル変数領域の確保がここで行われます。関数ごとの独立した実行環境を作るための入り口だと考えるとわかりやすいです。

エピローグ

関数末尾でスタックフレームを片付ける命令列です。スタックポインタとベースポインタを元へ戻し、`ret` で呼び出し元へ制御を返します。`return` を共通エピローグへ集める設計は、実装をかなり単純にします。

フォールスルー

`switch` 文で、ある `case` の末尾から次の `case` へそのまま流れる挙動です。C では明示的に `break` しない限り発生します。見落としやすい仕様ですが、コード生成では重要な挙動です。

変換単位

コンパイルの入力となる 1 つのソース単位です。簡単に言えば、プリプロセス後のソースファイルだと考えるとよいです。トップレベル宣言やグローバル変数、関数定義は変換単位の中で解釈されます。

デケイ

配列が式脈絡で先頭要素へのポインタへ崩れる現象を、英語では array decay と呼びます。配列とポインタが混同されやすい理由の 1 つです。`sizeof` や宣言時には崩れない、という点が重要です。

シンボル

名前付きの実体を指す一般的な言葉です。関数名、グローバル変数名、ラベル名などが典型です。コンパイラとリンカの両方で頻出する概念なので、早い段階から「名前が最終的に何を指すか」を意識しておくと理解しやすくなります。

付録 12: 追加練習課題

最後に、本書を読み終えたあとで手を動かすための課題をいくつか挙げます。どれも「次に何をやれば理解が深まるか」を意識した題材です。全部をやる必要はありませんが、1 つでも取り組むと理解がかなり定着します。

課題 1: AST ダンプを整える

各ノードの種類と子ノードだけでなく、型情報や左辺値性も表示する AST ダンプを作ってみてください。後半の機能追加で、内部状態を目で追えることのありがたさがよくわかります。

課題 2: エラーメッセージへ行番号を入れる

ファイル名と行番号、できれば列位置まで表示できるようにしてみてください。コンパイラの使い勝手が一気に良くなり、デバッグ速度が大きく変わることを実感できるはずです。

課題 3: `for` を `while` へ展開して考える

`for` 文のコード生成を、まずは内部的に `while` 相当へ変換してから実装する形にしてみてください。構文の違いと内部表現の違いを分けて考える練習になります。

課題 4: 構造体のアラインメントを可視化する

いくつかの `struct` 定義に対して、各メンバのオフセットと全体サイズを表示するデバッグ機能を作ってみてください。パディングの存在が一気に具体的になります。

課題 5: `switch` のテストを増やす

`case` の fall-through、`default`、`break` ありなしをそれぞれ分けたテストを用意してみてください。制御構文が「見た目より多くの状態を持っている」ことが見えてきます。

課題 6: 複合代入の副作用を確認する

`*p += 1` や `a[i]++` のような入力で、左辺が 1 回しか評価されていないことを確認するテストを書いてみてください。後半の難しさがどこにあるのかを実感しやすい課題です。

課題 7: 既存 C コードの小片を通す

短い文字列処理やトークナイズ処理など、現実の C コードから 20 行前後の断片を持ってきて、自分のコンパイラへ通るよう調整してみてください。人工的なサンプルでは見えなかった不足機能がよく見えます。

課題 8: 型エラーを少し厳密にする

ポインタどうしの加算や、存在しない構造体メンバ参照など、明らかな誤りだけでも診断できるようにしてみてください。意味解析を別フェーズへ切り出す価値がよくわかる課題です。

課題 9: セルフホスト準備として構造体を増やす

トークン、型、ノード、ローカル変数情報などを C の `struct` で表した小さなコードを書き、それが自分のコンパイラで通るかを確認してみてください。セルフホストへ近づく実感が得られます。

課題 10: 実装履歴を説明資料にする

Git の履歴やテストケースを使って、自分のコンパイラがどの順序で育ったかを簡単にまとめてみてください。人へ説明できる形にすると、自分の理解の抜けもよく見えます。

付録 13: よくある質問

最後に、読み進める途中で出やすい疑問を短い Q&A としてまとめます。本文を読んでいてひっかかったときの補助線として使ってください。

なぜ最初から最適化を考えないのですか

最初の段階で最適化まで入れると、正しさと効率の問題が混ざって見通しが悪くなります。本書の目的は、まず言語を正しく翻訳する骨格を作ることです。最適化は、その骨格が安定してから考えたほうが圧倒的に扱いやすくなります。

なぜアセンブリ出力なのですか

人間が目で読めるからです。機械語やオブジェクトファイルを直接吐くより、アセンブリのほうが生成結果を観察しやすく、失敗時の切り分けも楽です。教材としての透明性を重視すると、最初はアセンブリ出力がもっとも扱いやすい選択になります。

なぜ前半では型チェックを厳密にしないのですか

前半では、まず必要な型情報が内部表現として持てることが重要だからです。厳密な型エラー検出は有益ですが、早い段階でそれに力を使いすぎると、機能追加の速度が大きく落ちます。後半で内部表現が育ってから強化したほうが効率的です。

ポインタはなぜ難しく感じるのですか

値と場所を同時に意識しなければならないからです。整数だけの世界では「式は値を返す」と考えていてもかなり進めます。しかしポインタが入ると、「この式は場所を表すのか」「値へロードすべきか」を区別する必要が出ます。この切り替えが、最初の大きな壁になります。

構造体を入れると何が嬉しいのですか

コンパイラ自身の内部データを自然に表現できるようになります。トークンや型やノードを、ばらばらの配列ではなく、意味のまとまりとして扱えるようになることが大きいです。セルフホストを考えると、構造体はかなり早い段階で欲しくなります。

セルフホストは必須ですか

必須ではありません。セルフホストは強い到達点ですが、それ自体が唯一の価値ではありません。自分で字句解析、構文解析、型、コード生成を組み立てられるようになることだけでも十分に大きな成果です。セルフホストはその理解を試す 1 つの大きなテストだと考えるのが自然です。

実装が汚くなってきたらどうすればよいですか

機能追加が一区切りしたところで、小さくリファクタするのが有効です。特に、左辺値処理、型生成、ラベル管理、スコープ管理のような共通処理は、後からまとめ直す価値があります。コンパイラ実装では、動くものを先に作り、その後で共通化する流れがよく合います。

何を読めるようになれば次へ進んでよいですか

各段階で「生成されたアセンブリをざっと見て、値の流れを説明できる」状態になっていれば十分です。すべての命令を暗記する必要はありません。入力の構造が AST を経由して命令列へ落ちていることが追えれば、次へ進む準備はできています。

付録 14: 次に読むなら

本書を読み終えたあと、さらに先へ進みたいなら、方向は大きく 3 つあります。1 つ目はプリプロセッサや `union`、浮動小数点数など、C の未実装部分を埋めて実装を実用寄りにする道です。2 つ目はセルフホストへ向かい、自分のコンパイラ自身を通せるだけの表現力と安定性を育てる道です。3 つ目は別の言語実装や VM、JIT へ進み、ここで得た感覚を別の実行系へ持ち込む道です。

どの道へ進むにしても、本書で繰り返し使った「小さく作る」「中間結果を観察する」「必要になったら内部表現を育てる」という姿勢はそのまま通用します。コンパイラ作成で得られる一番大きな財産は、特定言語の仕様知識だけではなく、複雑な処理系を段階的に理解して組み立てる習慣そのものです。

参考資料

- Rui Ueyama, `compilerbook`
- Rui Ueyama, `9cc`
- Rui Ueyama, `chibicc`
- System V AMD64 ABI
- ELF specification
- Dragon Book などのコンパイラ教科書

索引

- ABI: 関数呼び出し規約や実行形式に関する外部との約束
- AST: 抽象構文木
- BNF: 文法記述のための代表的な記法
- ELF: Unix 系で一般的な実行形式
- left-value: 書き込み可能な場所を表す式
- parser: 字句列から構文木を作る部分
- tokenizer: 文字列をトークンへ分解する部分
- type: 式や宣言の意味を支える内部情報